Introduction to Deep Neural Networks (DNNs)

A Webinar from Pilot - UK AI Factory Antenna

Dr Charaka Palansuriya

EPCC, The University of Edinburgh

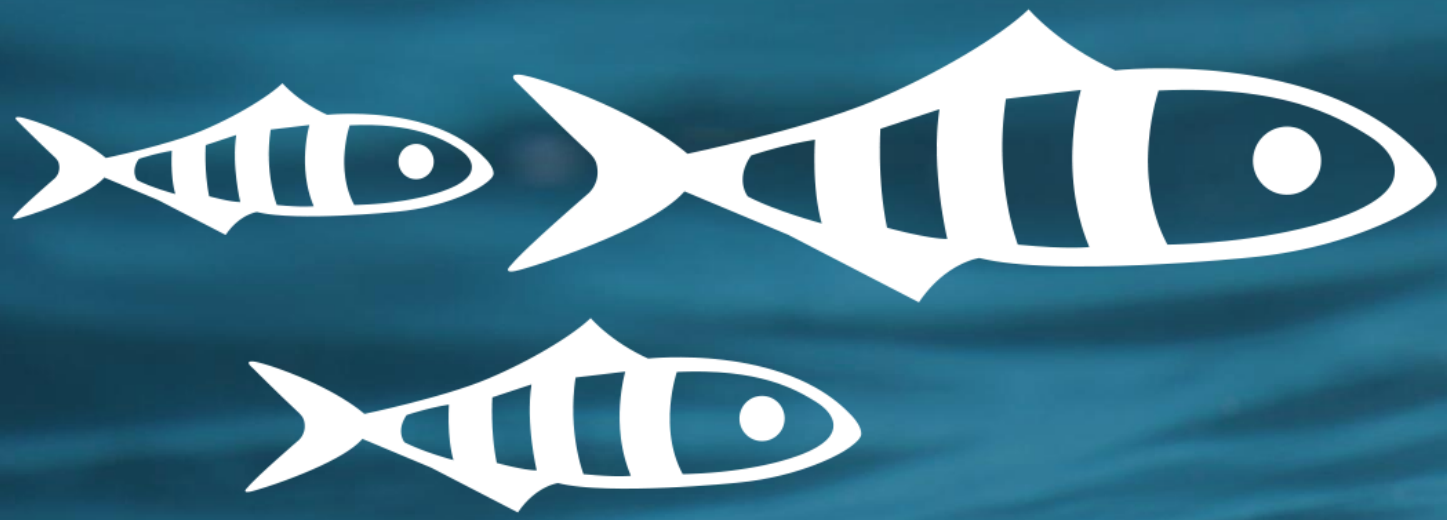
7th September, 2026, 11:30 BST

Pilot UK AI Factory Antenna

EuroHPC Joint Undertaking

Co-funded by the European Union

Department for Science, Innovation & Technology



Pilot

UK AI Factory Antenna

epcc

UK National Supercomputing Centre

EuroHPC Joint Undertaking

Co-funded by the European Union

Department for Science, Innovation & Technology

HammerHAI

HLRS High Performance Computing Center | Stuttgart

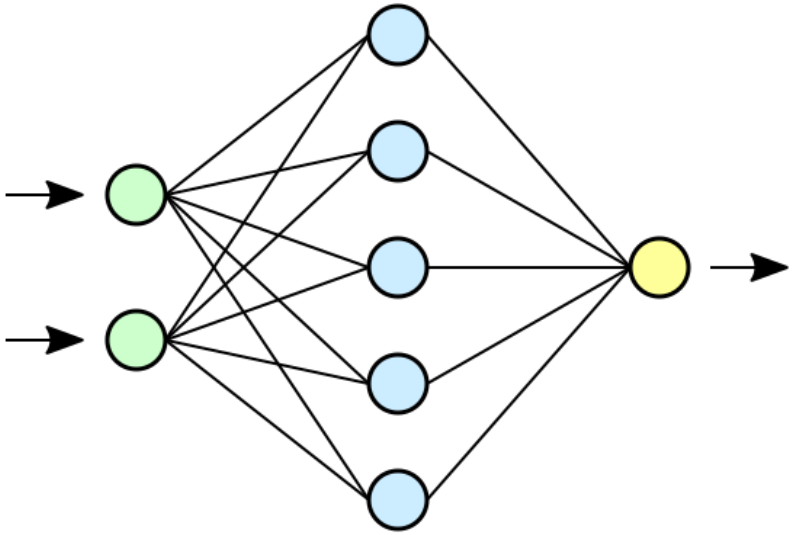
Introduction to DNNs

- Recap of what a Neural Network (NN) is
- What is a Deep Neural Network (DNN)?
- Main architectures of DNNs
- Feedforward Neural Networks (FNNs)
  - How a DNN works in the context of FNN
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs)
- Transformers
- Key Takeaways

Pilot
UK AI Factory Antenna

Recap: What is a Neural Network?

- A computational model that mimics structure and function of a human brain
  - Also referred to as Artificial Neural Networks (ANNs)
    - Or simply as Neural Networks (NNs)
  - Network structure consists of interconnected nodes (neurons)
    - Organised in layers
  - NN process information in one direction
  - Each node performs a function that can have multiple inputs and an output
  - NN performs a series of functions
    - Multiple layers mean it can compute complex functions by combining many simpler functions



Pilot
UK AI Factory Antenna

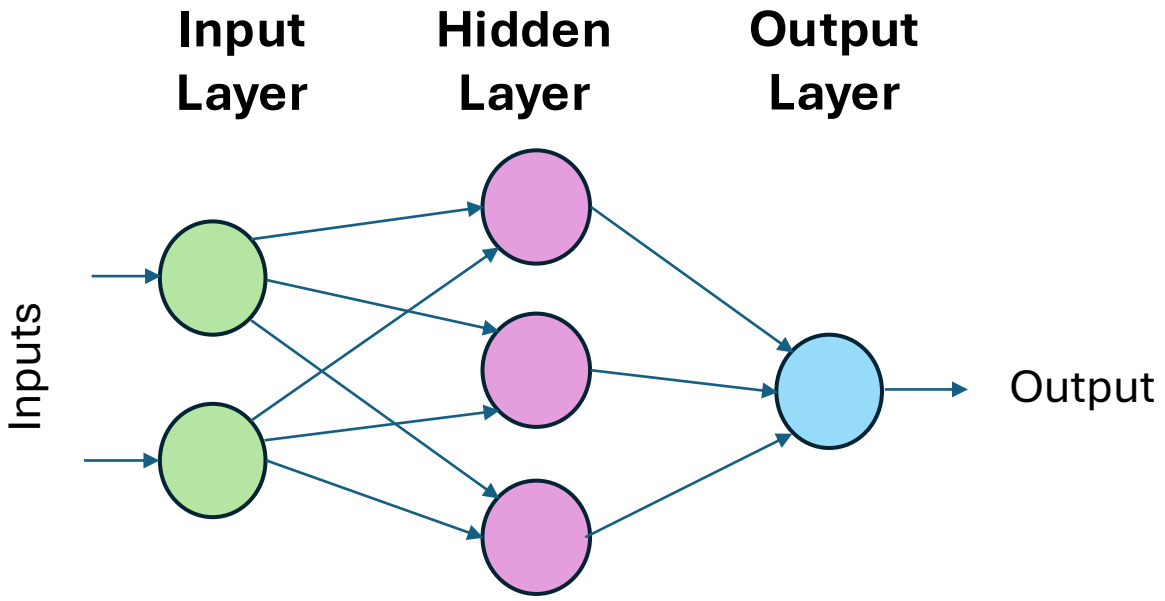
Recap: Motivation for creating NNs

- Works in a similar manner to how a human brain works making them powerful tools for solving complex problems
- They are more capable of independently learning and generalise what it learnt from data compared to other machine learning techniques
  - Neural networks can readily adapt to changing patterns of the data without requiring constant adjustments
- Suitable for learning non-linear complex hidden patterns
  - models highly non-linear systems where the relationships among variables are complex or unknown
- Can inference context in the data and suggest predictions based on this
  - For example, Transformer model has shown to understand the context of textual document or a conversation

Pilot
UK AI Factory Antenna

Recap: Simple example of a Neural Network

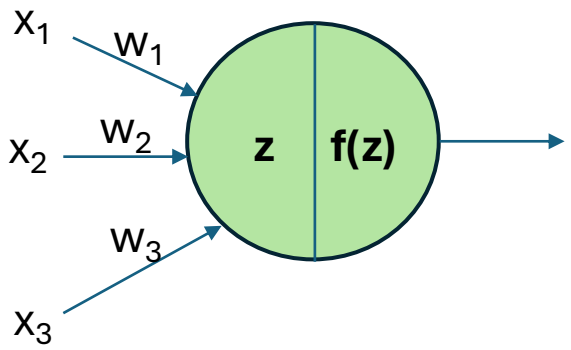
- Three types of layers:
  - Input layer
  - Hidden layer(s)
  - Output layer



Pilot
UK AI Factory Antenna

Recap: A closer look at how a neuron works

- Calculate $z = \sum_{i=1}^{n} w_i x_i + b$
  - Bias, b, allows calculations to be adjusted independent of the inputs
- Apply activation function f(z)
  - Common activation functions are shown in the next slide
  - This provides non-linearity to the neuron
- Output of the neuron is the result of applying the activation function to the function z

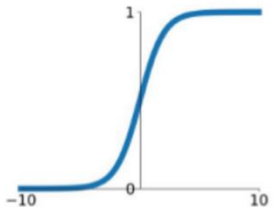


Pilot
UK AI Factory Antenna

Recap: Common Activation Functions

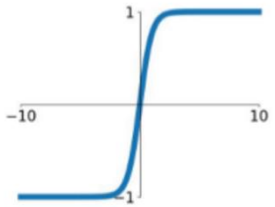
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



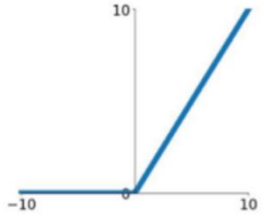
tanh

$$\tanh(x)$$



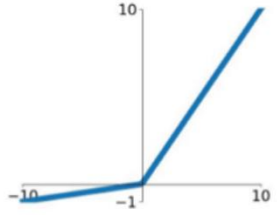
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

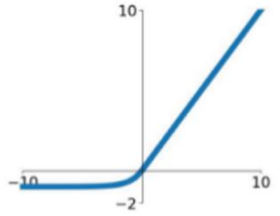


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

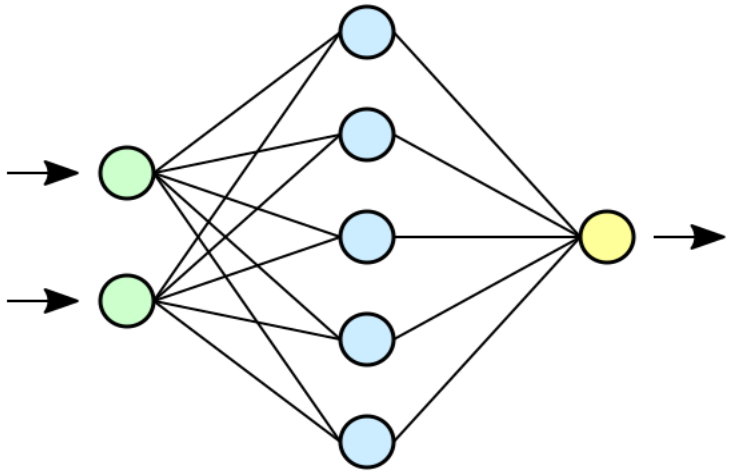


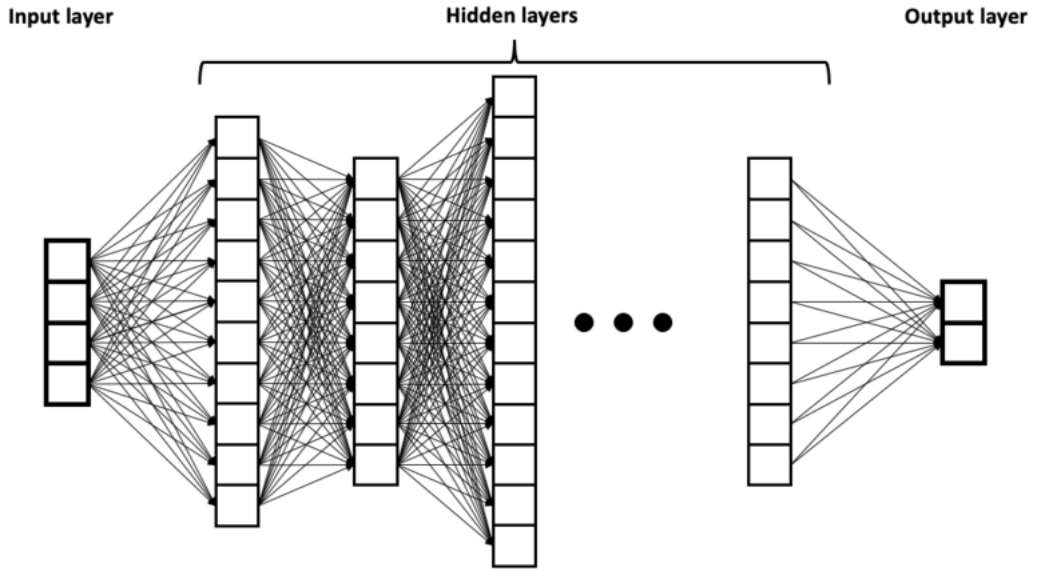
From https://towardsdatascience.com/complete-guide-of-activation-functions-34076e95d044

Pilot
UK AI Factory Antenna

What is a Deep Neural Network (DNN)?

- A **Deep Neural Network** is simply a neural network with many hidden layers





- And **Deep Learning** is simply Machine Learning using Deep Neural Networks
  - Some people use the terms "Deep Learning" and "Neural Networks" almost synonymously

Pilot
UK AI Factory Antenna

What is a DNN – continued.

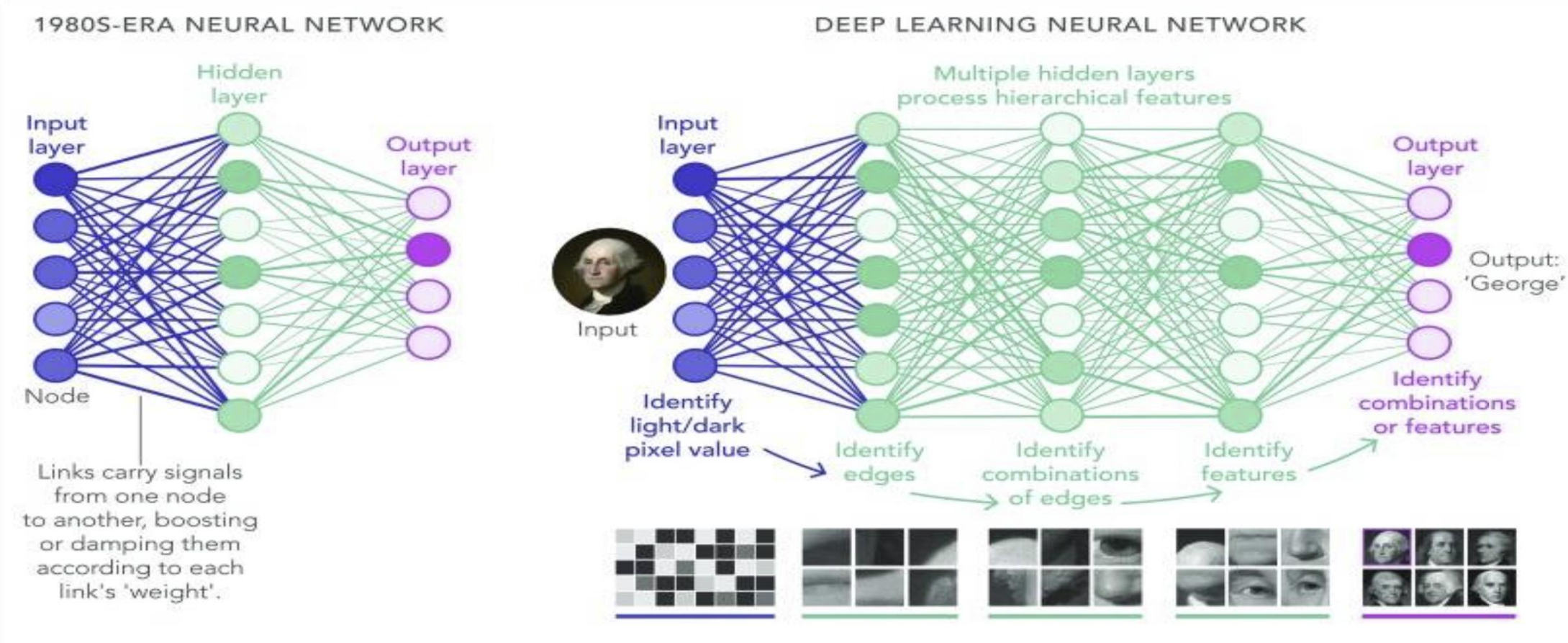
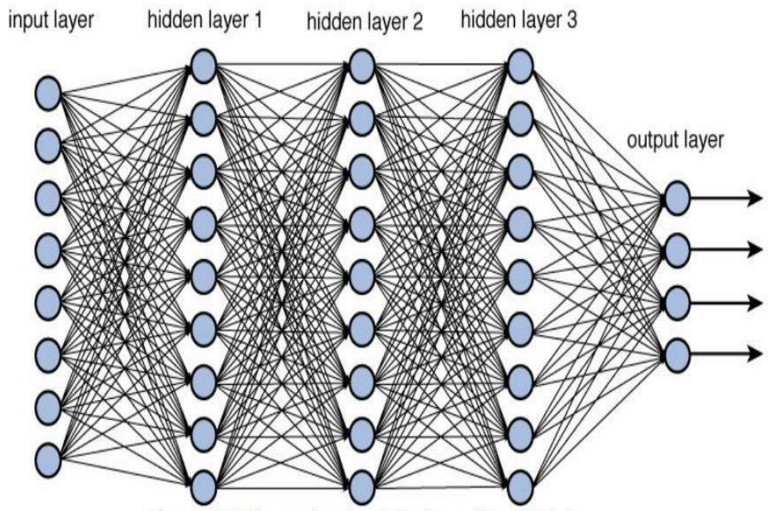


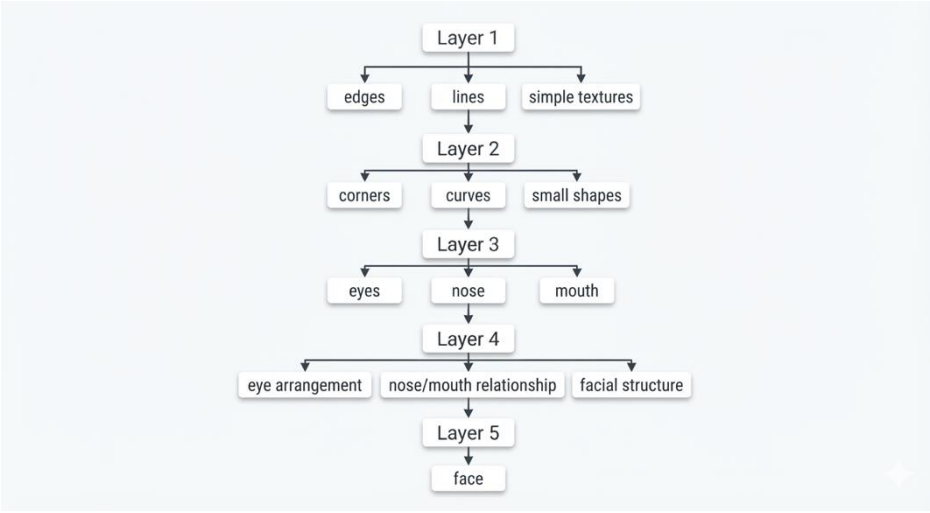
Image from https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6347705/

Pilot
UK AI Factory Antenna

What is a DNN – continued.

- DNN have multiple layers
  - This is referred to as the **depth.**
  - Hidden layer capture data features.
- Each layer can have variable numbers of neurons
  - This is referred to as the **width.**
- Specific configuration/construction of layers dependent on network architecture and problem domain.
- Depth and width of a DNN impacts computational cost, memory as well as amount of training data requirements.
- DNNs work well for problems with hierarchical structures
  - E.g., Recognising a face (see diagram).





Pilot
UK AI Factory Antenna

DNN: Use Cases

- Medical diagnosis
  - cancer diagnosis
  - processing patient data to determine probable diagnoses
- Self-driving Cars
  - E.g., machine vision using Convolutional Neural Networks (CNNs)
- Language translation
  - Document translation between two languages
- Speech recognition and translation
  - Automated speech to text writing
  - Real time speech translation between two languages
- Synthetic image and video generation
  - DALL-E 2
  - Stable Videos Diffusion
- AI Assistance + AI Agents
  - ChatGPT (from OpenAI)
  - Claude (from Anthropic)
  - Gemini (from Google)

Pilot
UK AI Factory Antenna

Main architectures of DNNs

- Feedforward Neural Networks (FNNs)
  - Usef for classification and regression.
- Convolutional Neural Networks (CNNs)
  - Mainly used for image recognition.
- Recurrent Neural Networks (RNNs)
  - Suitable for sequential data processing like sentiment analysis and language translations.
- Transformer architecture
  - General purpose usage
    - Including Natural Language Processing (NLP), Image Processing, etc.

Pilot
UK AI Factory Antenna

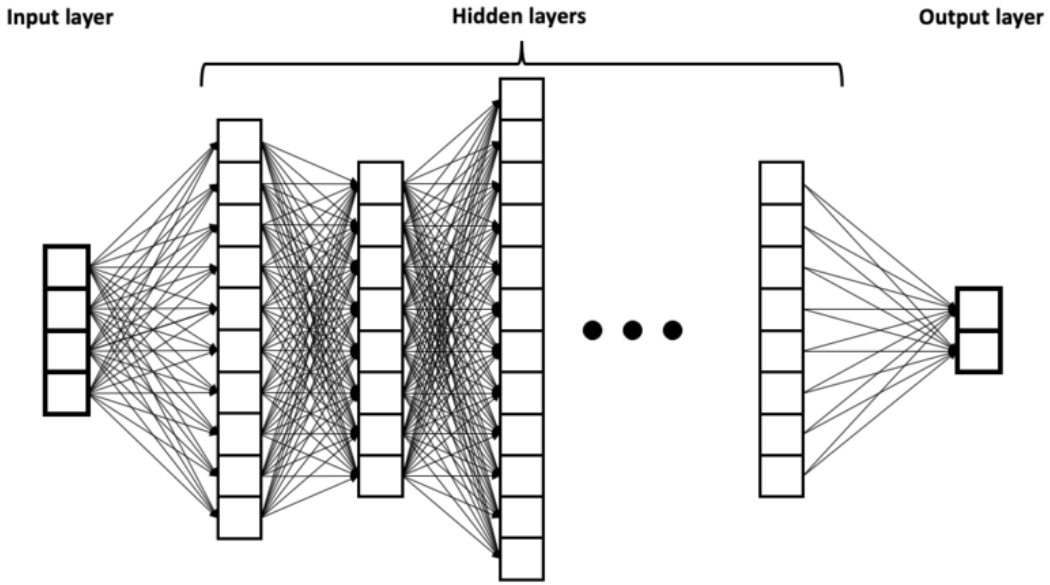
FeedForward Neural Network (FFNs)

- Also known as:
  - Fully Connected NN
  - MultiLayer Perceptron (MLP)
- One of the simplest types NN in use
- Used for classification and regression
- A fundamental architecture component that goes into other more complex NN architectures
  - E.g., Transformers, CNNs and RNNs use FeedForward NN
- Lets have a deeper look at how a FeedForward NN work

Pilot
UK AI Factory Antenna

Lets look at how a simple FNN works

- Three types of layers:
  - Input layer
  - Hidden layer(s)
  - Output layer



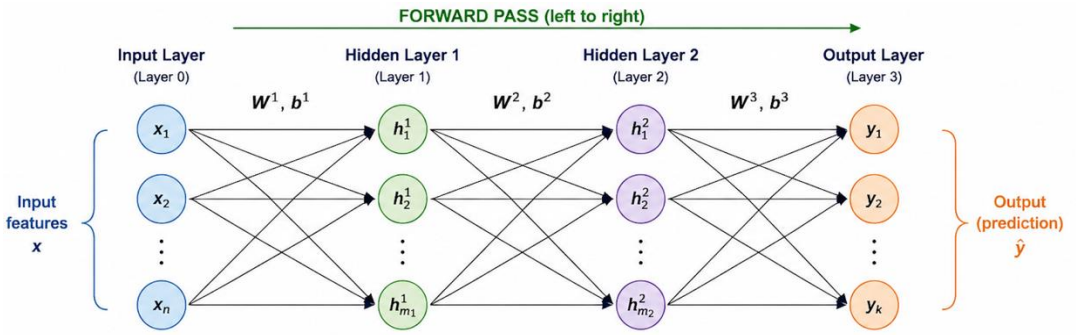
Pilot
UK AI Factory Antenna

Steps in a FNN

- **Forward propagation**
  - Step 1: First hidden layer receives data from the input layer:
    - performs its computations
      - Weighted sum of inputs + biases
    - Applies the activation function
  - Step 2: Activated connections pass results to the next layer:
    - Performs the weighted sum computations as above
    - Applies the activation function
    - Above is repeated for all hidden layers until the output layer reached
  - Step 3: Output layer computes and produces the network output
  - Step 4: Calculate Loss
    - Compare the network outputs with true target values
    - Compute the loss using a loss function

Flow of Information

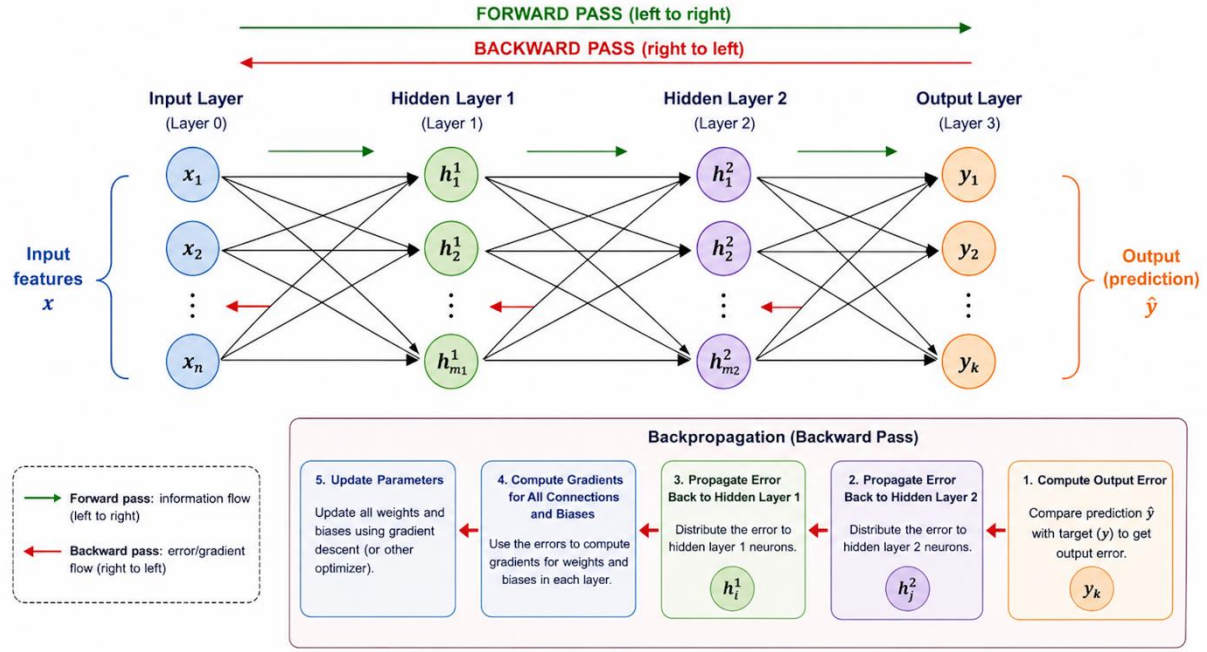
Input features → Hidden Layer 1 → Hidden Layer 2 → Output (prediction)

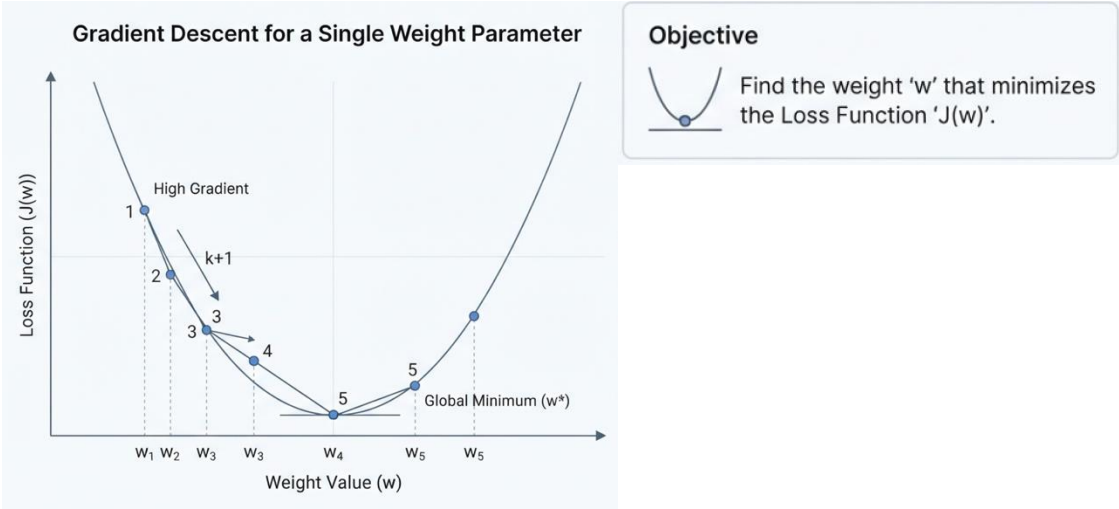


Pilot
UK AI Factory Antenna

Steps in a FNN – continued.

- Step 5: **Backpropagation**
  - Compute the gradients of the loss (error) function with respect to each parameter
  - Propagate the gradients backwords, layer-by-layer
  - Update the parameters in the direction that minimises the loss
    - i.e., minimise the difference between network output and the true target values
    - Use an optimisation algorithm such as Gradient Descend algorithm
- Step 6: Repeat
  - Iterate over the dataset multiple times (epochs)
    - Adjust parameters at each iteration to minimise the loss
    - Continue until convergence criteria is met for the training data
    - **Then the architecture is considered a trained model with learned parameters**.
      - i.e., this FNN model is now ready for inferencing work.





Pilot
UK AI Factory Antenna

Choosing a loss function

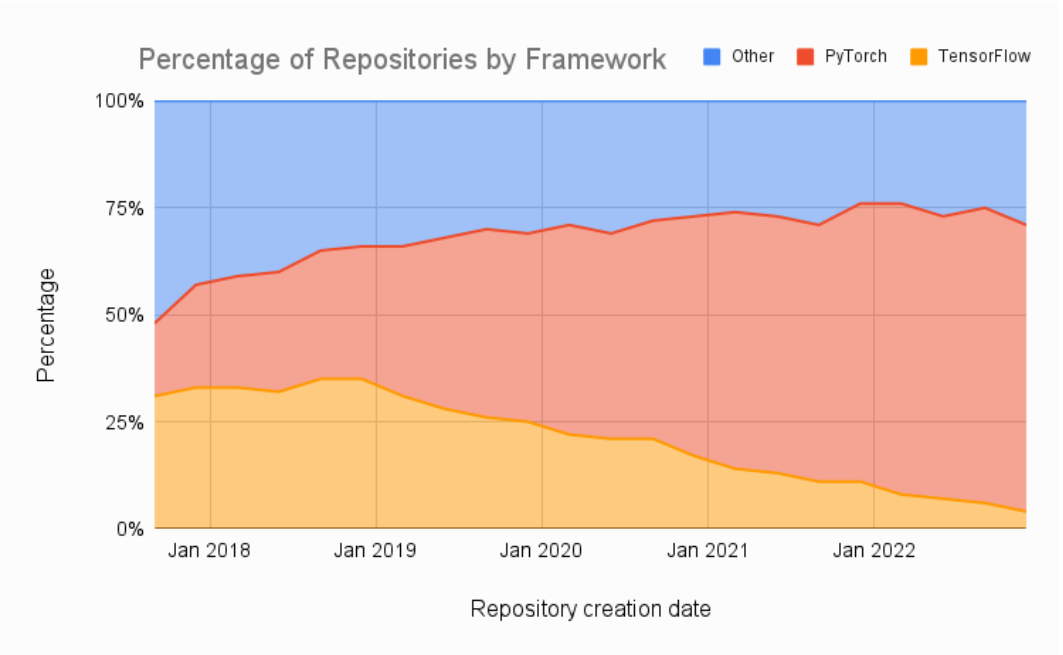
- Regression type problems
  - Mean Squared Error (MSE)
    - Average squared difference between the predicted and true values
  - Mean Absolute Error (MAE)
    - Average absolute difference between the predicted and the true values
- Binary Classification
  - Binary Cross-Entropy Loss
    - Measures the dissimilarity between the true binary labels and the predicted probabilities
- Multi-Class Classification
  - Categorical Cross-Entropy Loss
    - Measures the difference between true class distributions and the predicted probabilities
  - Sparse categorical Cross-Entropy Loss
- Sequence/ Language modelling and machine translation
  - Categorical Cross-Entropy Loss
    - Measures the dissimilarity between the predicted and true probability distributions of sequences

Pilot
UK AI Factory Antenna

FNN using PyTorch

- What does it look like building a FNN using PyTorch?

- Why PyTorch used here?
  - PyTorch has become the most popular ML framework in use by researchers
  - All major AI accelerated hardware vendors have ported PyTorch to their platforms; e.g.,
    - NVIDIA
    - AMD
    - Apple
    - Cerebras



- Image from https://www.assemblyai.com/blog/pytorch-vs-tensorflow-in-2023/

Pilot
UK AI Factory Antenna

Building a FNN in PyTorch

```
class FeedForwardNN(nn.Module):
  """
  Class initialisation method defines feedforward neural
  network architecture
  """
  def __init__(self, input_dim, hidden_dim, output_dim):
    super(FeedForwardNN, self).__init__()
    # Add a fully connected layer from input to hidden
    self.fc1 = nn.Linear(input_dim, hidden_dim)
    # Use ReLU non-linear activation function
    self.relu = nn.ReLU()
    # Fully connected layer from hidden to output
    self.fc2 = nn.Linear(hidden_dim, output_dim)

  """ defines the forward pass of the neural network """
  def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    return x
```

Pilot
UK AI Factory Antenna

PyTorch Training Loop

```
def train_model(sample_data, labels, model, criterion, optimizer):
  """
  Training loop for the neural network
  """
  num_epochs = 1000

 for epoch in range(num_epochs):
    # First perform the forward pass
    predictions = model(sample_data)
    # Then compute the loss
    loss = criterion(predictions, labels)
    # Reset the gradients to zero (by default PyTorch accumulates gradients)
    optimizer.zero_grad()
    # Use backpropagation to calculate the gradients
    loss.backward()
    # Update the weights in direction that minimises the loss
    optimizer.step()
```

Pilot
UK AI Factory Antenna

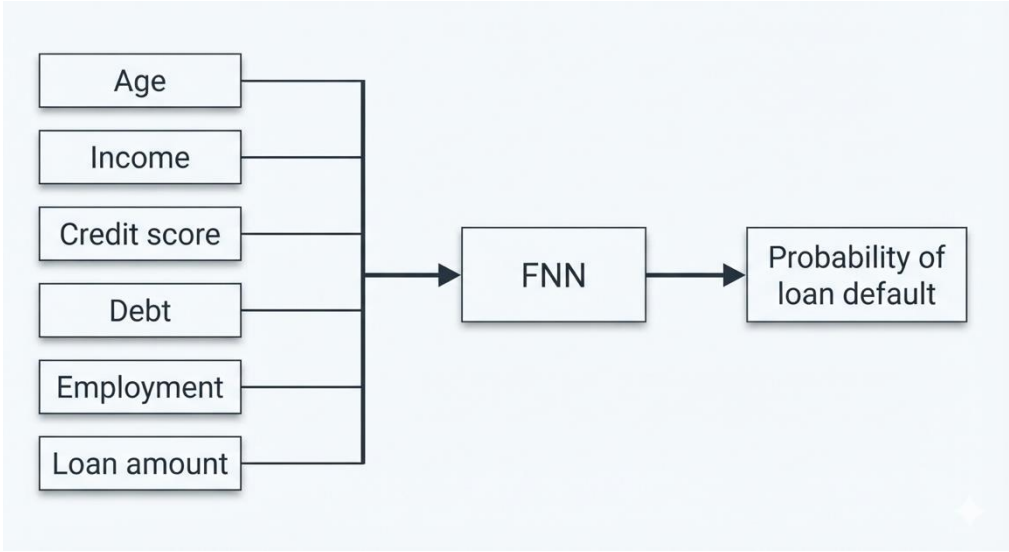
PyTorch main function

```
def main():
 """
 Sets up NN configuration and training data, then trains the model
 and tests it with new data
 """
 # Sample data for training the neural network
 input_dim = 2 # Two input values
 hidden_dim = 10 # Ten neurons in the hidden layer
 output_dim = 1 # One output value (the sum of two input values)
 sample_data = torch.tensor([[1.0, 2.0], [2.0, 3.0], [3.0, 4.0], [4.0, 5.0], [5.0, 6.0]])

 # Sum the input values in each row (i.e., dim=1)
 labels = torch.sum(sample_data, dim=1).unsqueeze(1) # Unsqueeze to match the shape of predictions
 print(f"Shape of labels = {labels.shape}")
 print(f"labels = \n {labels}")

# Create an instance of the neural network
 model = FeedForwardNN(input_dim, hidden_dim, output_dim)

# Use the Mean Squared Error loss function
 criterion = nn.MSELoss()

# Define the optimizer
 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
 train_model(sample_data, labels, model, criterion, optimizer)
```

FNNs: Pros

- FNNs are conceptually a simple architecture
  - Simple to implement, train and experiment with.

- Good at learning non-linear complex relationships, e.g.:
  - Financial prediction
  - Credit Scoring
  - Fraud detection
  - Medical risk prediction
  - Non-linear regression type problems
  - Etc.



Pilot
UK AI Factory Antenna

FNNs: Cons

- Not good at exploiting any structure in the data
  - E.g., in image recognition.

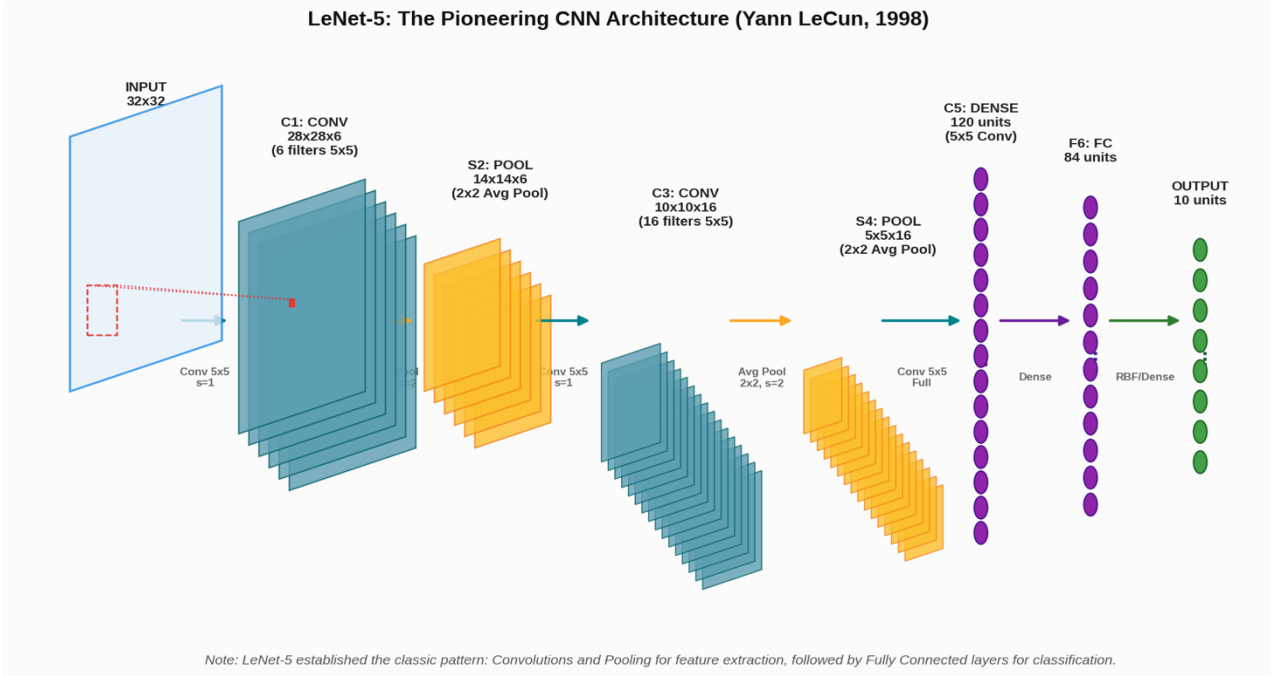
- Not good at sequential data
  - E.g., "I ordered a pizza but it was delivered late"
    - "it" requires information about the previous words
  - No "memory" of what was processed earlier by the network.

- Not good at processing variable length sequence
  - Fixed-size inputs are required.

Pilot
UK AI Factory Antenna

Convolutional Neural Networks (CNNs)

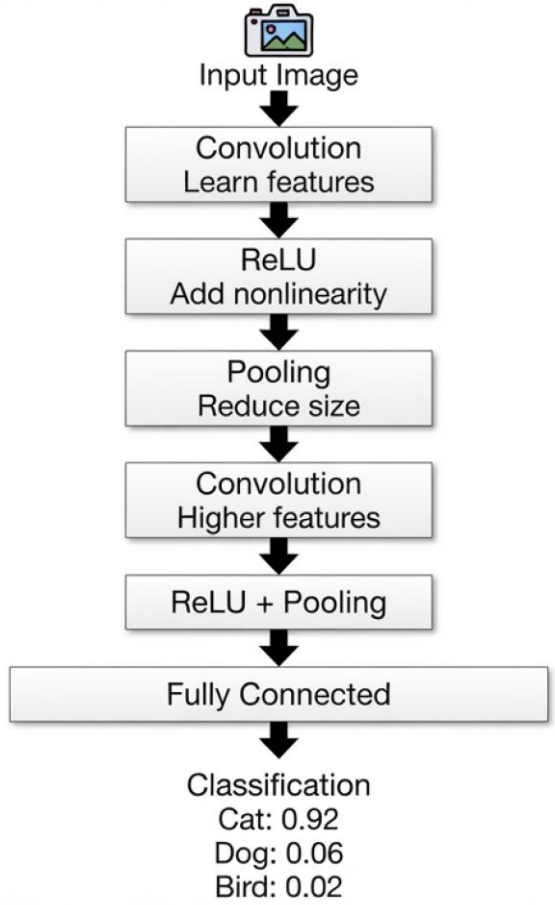
- This is a type of DNN that is mainly designed to work with grid-like data structures such as images.
- Images can contain millions of pixels
  - Something like FNN would need to connect every pixel to too many neurons
    - The network will be too expensive to train
    - Loses spatial information – e.g., does not realise nearby pixels are related.
      - CNNs exploit the spatial structure of images.
- Widely used for image classification, object detection, facial recognition, medical imaging, autonomous vehicles and many other computer vision tasks.



Pilot
UK AI Factory Antenna

CNNs Continued.

- CNNs have the following key components:
  - Convolutional Layers
    - Performs the most important operation in a CNN
    - A convolution uses a small matrix called a **filter** or **kernel** and slide it across the image.
      - This detects features such as edges, corners, shapes.
  - Pooling Layers
    - Pooling is used to reduce the spatial dimension.
    - Only need **approximately where a feature** is.
    - This also reduce computational cost and overfitting.
  - Non-linearity
    - Allows CNN to learn complex non-linear relationships.
  - Fully connected layers
    - serve as classifiers of these extracted features – assign probability



Pilot
UK AI Factory Antenna

Convolutional Layer: Convolving a 6x6 image with a 3x3 filter

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6 X 6 image

1	0	-1
1	0	-1
1	0	-1

3 X 3 filter

for vertical edges

=

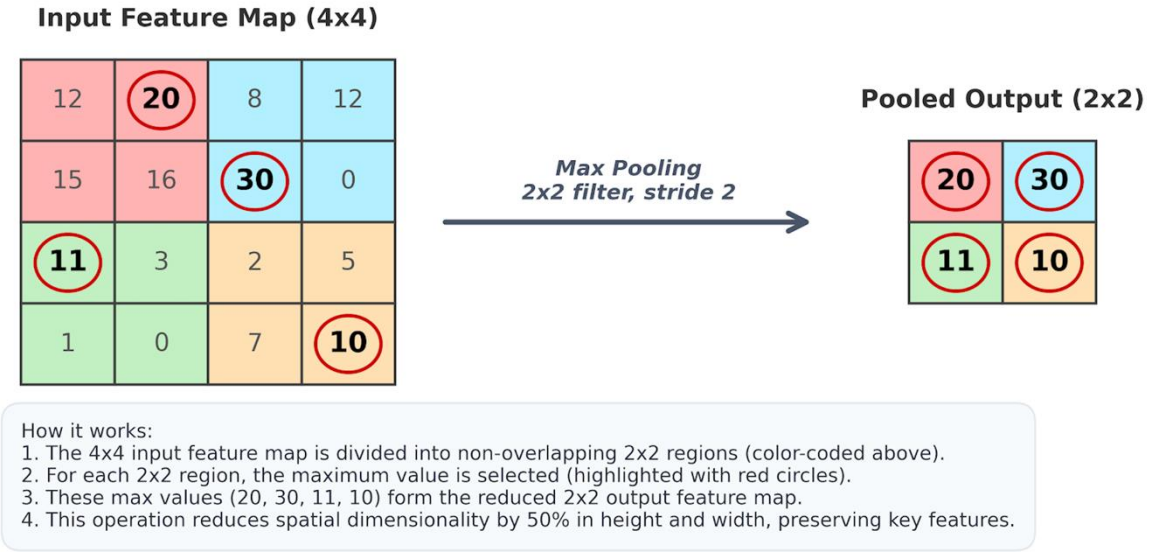
-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

- Dot product for first output element:
  - 3∗1 + 0*0 + 1∗(-1) + 1∗1 + 5∗0 + 8∗(-1) + 2∗1 + 7∗0 + 2∗(-1) = -5
- Type of filter helps to detect horizontal or vertical edges
- Higher-magnitude values in output matrix will represent areas where edges have been located

Pilot
UK AI Factory Antenna

Pooling Layer: Max Pooling

- Dimensionality Reduction:
  - downsampling to reduce spatial resolution of feature maps.
  - Max and Average are most common
- Stride-based Windows (dividing to local regions:
  - E.g., 2x2 pooling operation with a stride of 2:
    - the 4x4 input feature map is divided into 4 non-overlapping 2x2 pooling windows

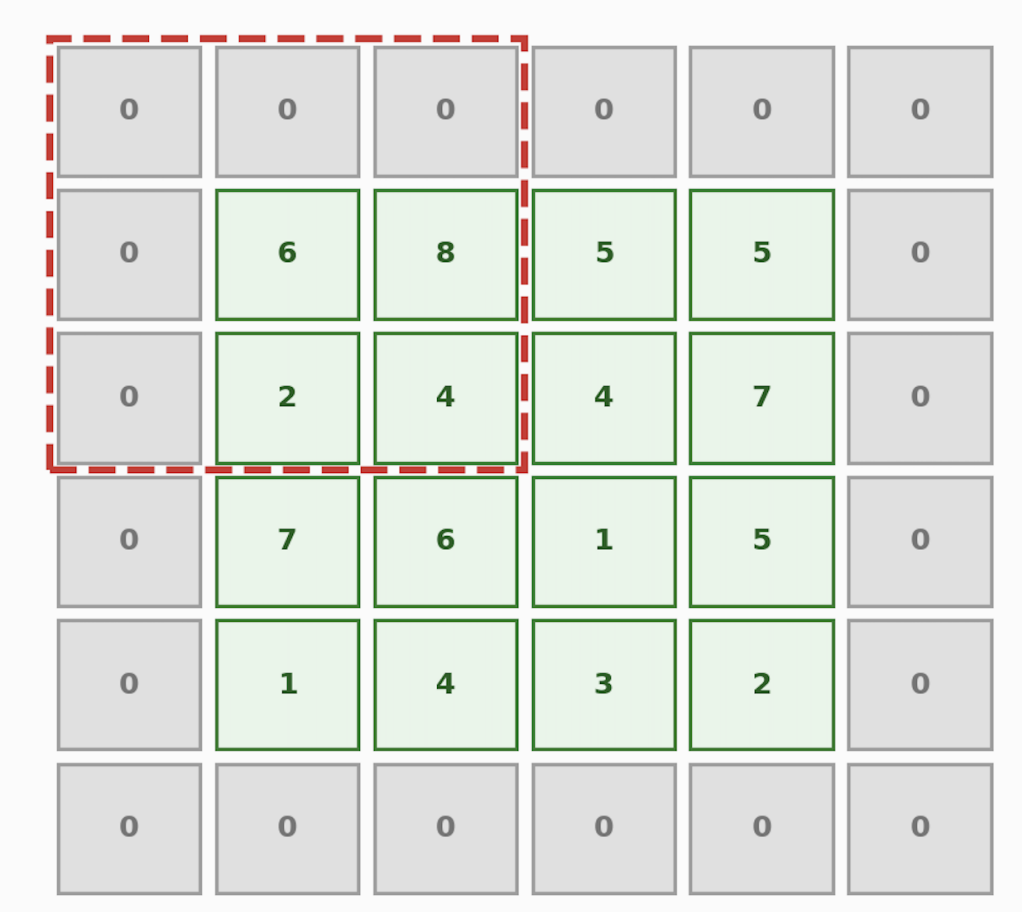


How it works:
1. The 4x4 input feature map is divided into non-overlapping 2x2 regions (color-coded above).
2. For each 2x2 region, the maximum value is selected (highlighted with red circles).
3. These max values (20, 30, 11, 10) form the reduced 2x2 output feature map.
4. This operation reduces spatial dimensionality by 50% in height and width, preserving key features.

Pilot
UK AI Factory Antenna

CNNs: Padding

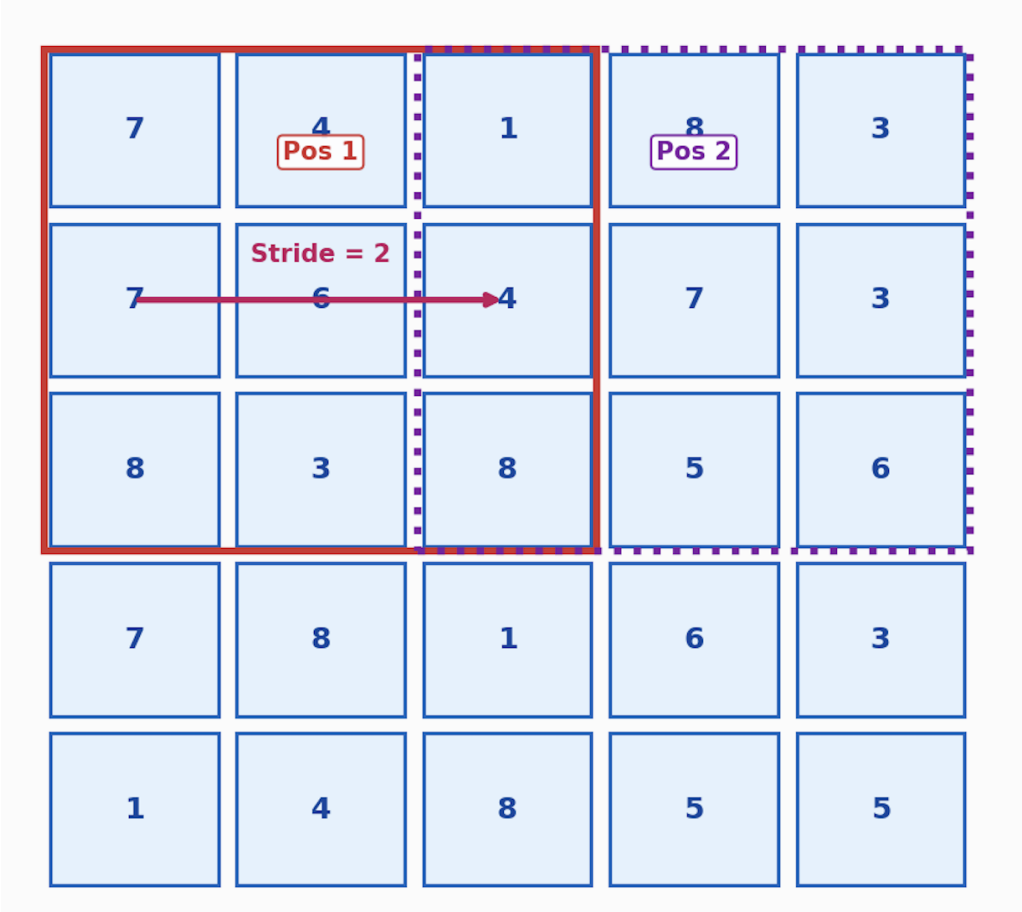
- Padding involves surrounding an input feature map (e.g., a 2D image) with extra border of synthetic pixels.
  - Usually filled with zeros
    - "Zero-Padding"
- Without padding:
  - a convolution makes the feature map smaller.
  - Pixels near the edges participate in fewer convolution operations than the pixels in the centre.



Pilot
UK AI Factory Antenna

CNNs: Stride

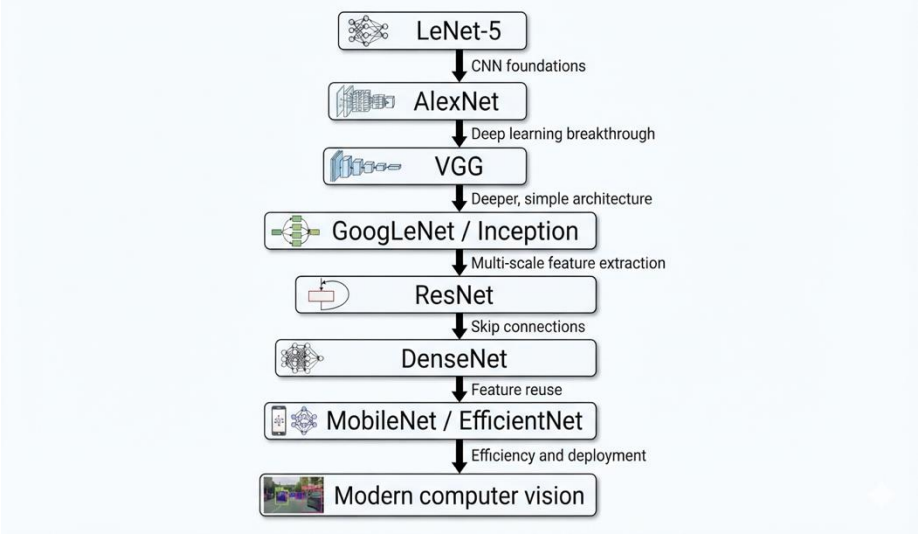
- Determines how far a convolutional filter moves across the input feature map at each step.
  - Stride greater than 1 is used to reduce the spatial size of the feature map.
    - Reduces computation
    - Gradually create more compact representations.
- CNNs often use Stride and paddings together.

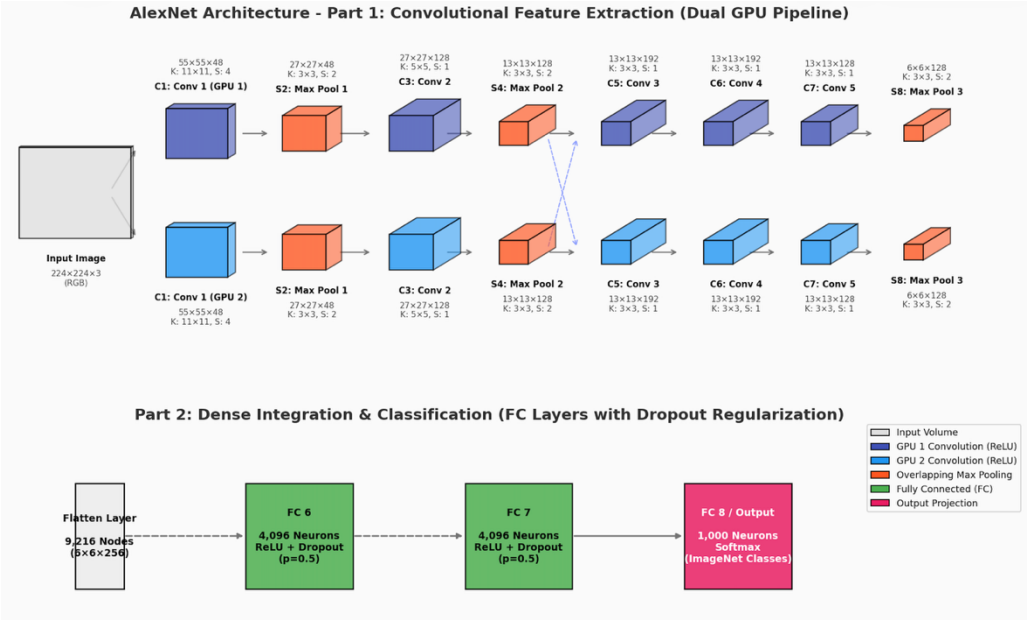


Pilot
UK AI Factory Antenna

CNNs: Well known architectures

- LeNet-5 (original CNN)
  - Foundational CNN developed for handwritten digit recognition.
- AlexNet (breakthrough CNN)
  - Major breakthrough in 2012
  - Shown a significant improvement in ImageNet image-classification competition.
  - Used a **much deeper network** than earlier CNNs.
  - ReLU activation.
  - Large ImageNet dataset.
  - GPU based training.





Pilot
UK AI Factory Antenna

CNNs: Pros

- Highly effective for classification, detection and segmentation of images.
- Parameter sharing – same filter is re-used across the image.
  - Reduced parameters compared to fully connected networks.
- Hierarchical and multi-scale feature extraction.
  - By stacking convolutional and pooling layers sequentially, CNNs learns hierarchy of features.
- Exploitation of the spatial structure/locality.
  - Unlike FNNs which would treat images as flattened and unstructured vectors, CNNs exploits adjacent/neighbour elements.
- Strong resistance to overfitting.
  - Shared weights in the filters forced more general feature extraction.
  - Pooling too help with generalizability as only the neighbourhood is detected not the exact position.

Pilot
UK AI Factory Antenna

CNNs: Cons

- Required a large amount of training data
  - Training from scratch requires a lot of labelled images.
  - Can overfit if not enough training data.
- Training can be time consuming
  - Large CNNs requires significant amount of GPUs/CPUs.
- Sensitive to changes in data distribution
  - Performance can fall when test images differs significantly from the training data.

Pilot
UK AI Factory Antenna

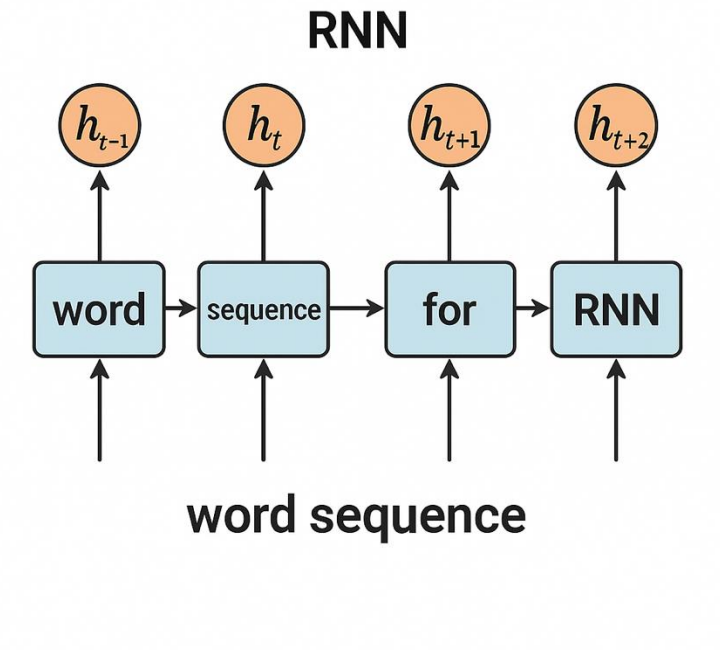
Recurrent Neural Networks (RNNs)

- Suitable for modelling sequence data.
- Suitable for modelling temporal dependencies
  - Many real-world tasks are inherently ordered and suitable for sequential processing
    - e.g., audio signals, time-series
- Simple and efficient for handling small tasks
  - RNNs are computationally lightweight and easy to implement
  - Suitable for low-latency, real-time or resource constrained platforms
    - e.g., Mobile devices, IoT devices, embedded system
- Smaller models with acceptable performance
  - When extreme accuracy of Transformers are not required RNNs give good enough results using far less computation
    - e.g., Anomaly detection, keyword spotting

Pilot
UK AI Factory Antenna

RNNs: What are Sequence Data?

- Elements in a sequence appear in certain order
  - Order in which the training data presented to the model is important
  - This is in contrast to many other machine learning algorithms where input is considered Independent and Identically Distributed (ID)
- Examples of sequence data:
  - Textual data like books, documents, etc.
  - DNA sequences
  - Speech recordings (time series)
  - Video recordings (time series)



Pilot
UK AI Factory Antenna

RNNs: Categories of sequence modelling

- One-to-one
  - Single input and single output
  - e.g., single image classification
- One-to-many
  - Single input and sequence output
  - e.g., single image captioning
- Many-to-one
  - Sequence input and single output
  - e.g., sentiment analysis of a sequence of text
- Many-to-many
  - Sequence input and sequence output
  - e.g., language translation

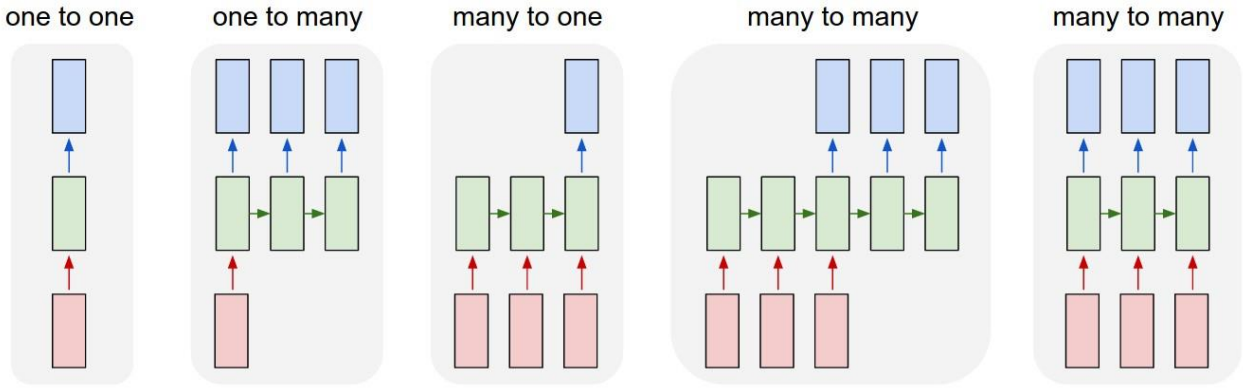
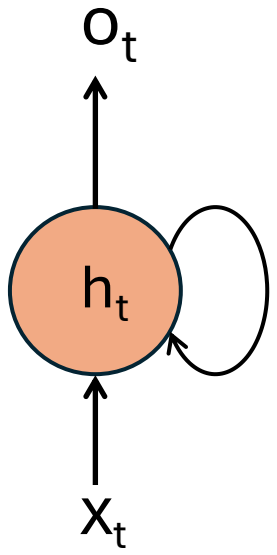


Image from https://karpathy.github.io/2015/05/21/rnn-effectiveness/

Pilot
UK AI Factory Antenna

RNNs

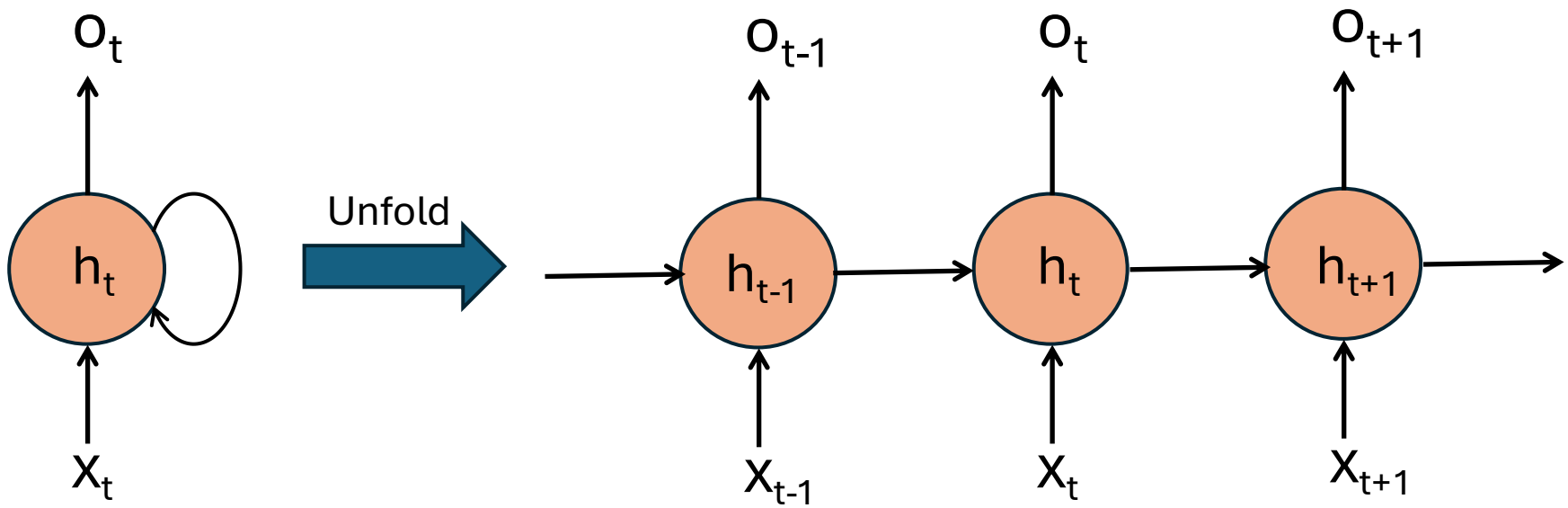
- RNNs are designed to model sequence data
- RNNs maintains an internal state ("memory"):
  - This internal state is referred to as the **hidden state**
  - The hidden state **summarises the information from previous time steps** and is used when processing new inputs.
  - At each time step, the hidden layer receives inputs from
    - The input at the current time step
    - The hidden state from the previous time step
      - i.e., Loop in the hidden layer
      - This functionality allows RNNs to have a memory of the past events
  - RNNs can have multiple hidden layers
    - Similar to deep FFNs.



Pilot
UK AI Factory Antenna

RNNs unfolded

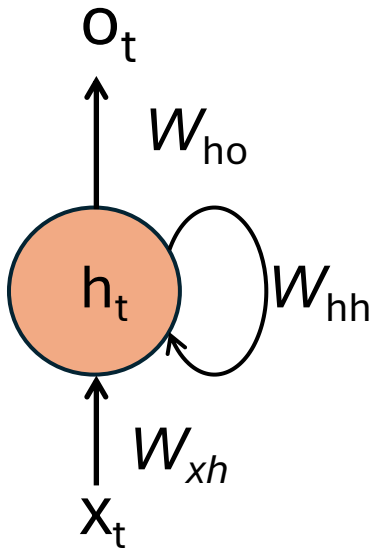
- A single layer RNN unfolded



Pilot
UK AI Factory Antenna

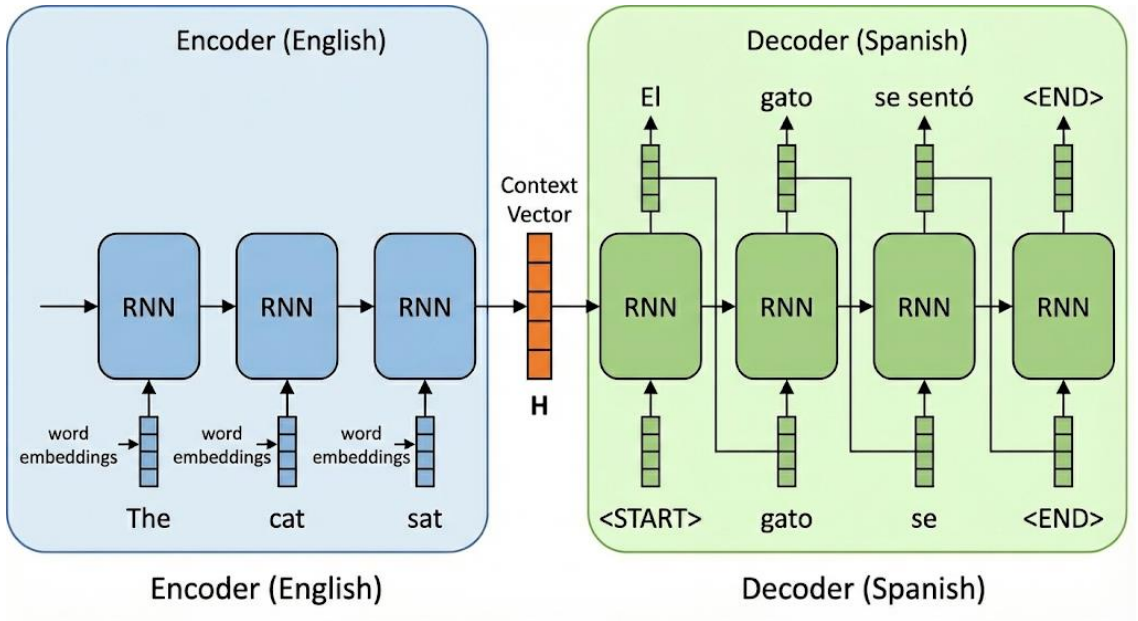
How an RNN hidden layer works

- Calculate $h_t = \sigma_h(w_{xh}x_t + w_{hh}h_{t-1} + b_h)$
  - w_{xh} and w_{hh} are learnable matrices
  - x_t is the input vector
  - h_{t-1} is the hidden state vector from the previous time step
  - b_h is the learnable bias vector of the hidden layer
  - σ_h is the activation function of the hidden layer
- Different non-linear activation functions could be used for σ_h
  - Common to use **tanh** function for the hidden layer
- Calculate $o_t = \sigma_o(w_{ho}h_t + b_o)$
  - Non-linear activation function σ_o depends on the task
    - Softmax could be used for multi-class classification
    - ReLu for regression type output



Pilot
UK AI Factory Antenna

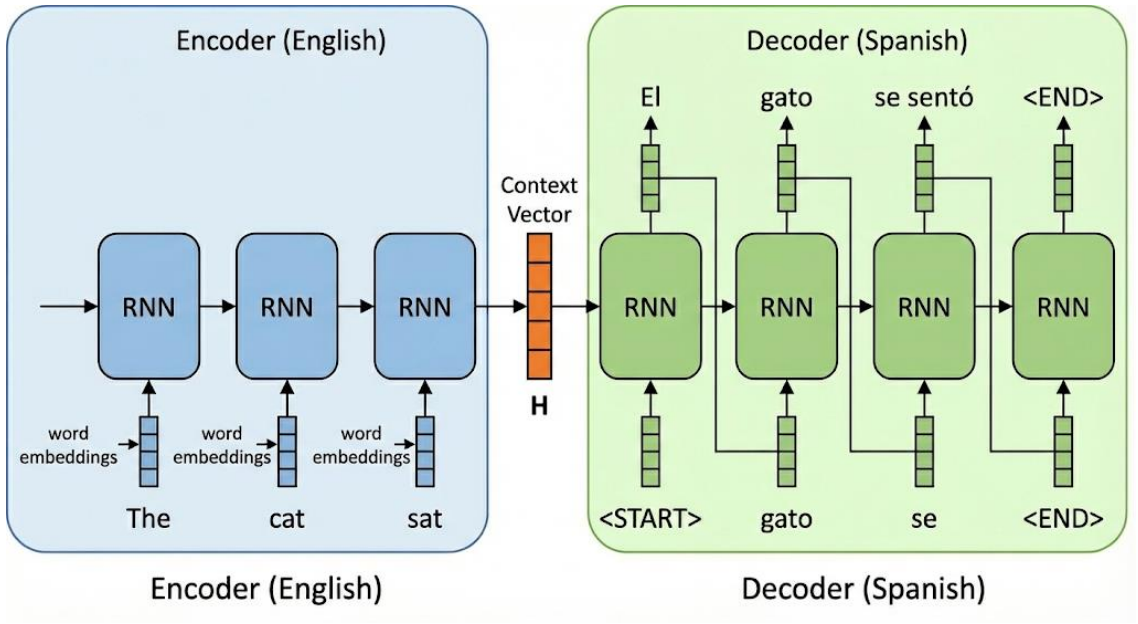
How an RNN hidden layer works – An example



- Lets look at using an RNN for translating an English sentence to a Spanish sentence.
- Translate English sequence **"The cat sat"** into the Spanish sequence **"El gato se sentó"**
- **Word Embeddings**
  - Each word is converted into a dense list of numbers (feature vector)
    - E.g., "cat" might become [0.14, -0.98, 0.5]
- **Encoder** reads the input sequence and summarise it
  - Read "The" then update the hidden state, h_t (its memory)
  - Read "cat" then combine this new information with its previous memory
  - Finally, read "sat" and update the memory one last time.

Pilot
UK AI Factory Antenna

How an RNN hidden layer works – An example continued



- **Context Vector**
  - This is the final hidden state (memory) following the computation in the encoder
  - This is a single vector numbers that represents the meaning of the entire sentence
  - Encoder passes this vector to the decoder
- **Decoder**
  - Takes the context vector and starts generating Spanish words one by one
  - Steps:
    - Input <START>: RNN looks at its state (at this initial state – same as the context vector) and output word with the highest probability: "El"
    - Input "El": use the previous output ("El") as the input for next step. Then update its state and predicts: "gato"
    - Input "gato": update the state and predicts "se"
    - Stop: when <END> is detected

Pilot
UK AI Factory Antenna

Limitations of a vanilla RNN

- Hard to learn long-range dependencies
  - Hidden state forgets older context
- Vanishing or exploding gradients due to backpropagation through time
  - Vanishing gradients -> can *not* learn long term dependencies
  - Exploding gradients -> Training becomes unstable
- Improved versions of RNNs were developed to handle above limitations
  - Long Short-Term Memory (LSTM)
  - Gated Recurrent Unit (GRU)

Pilot
UK AI Factory Antenna

Long Short-Term Memory (LSTM)

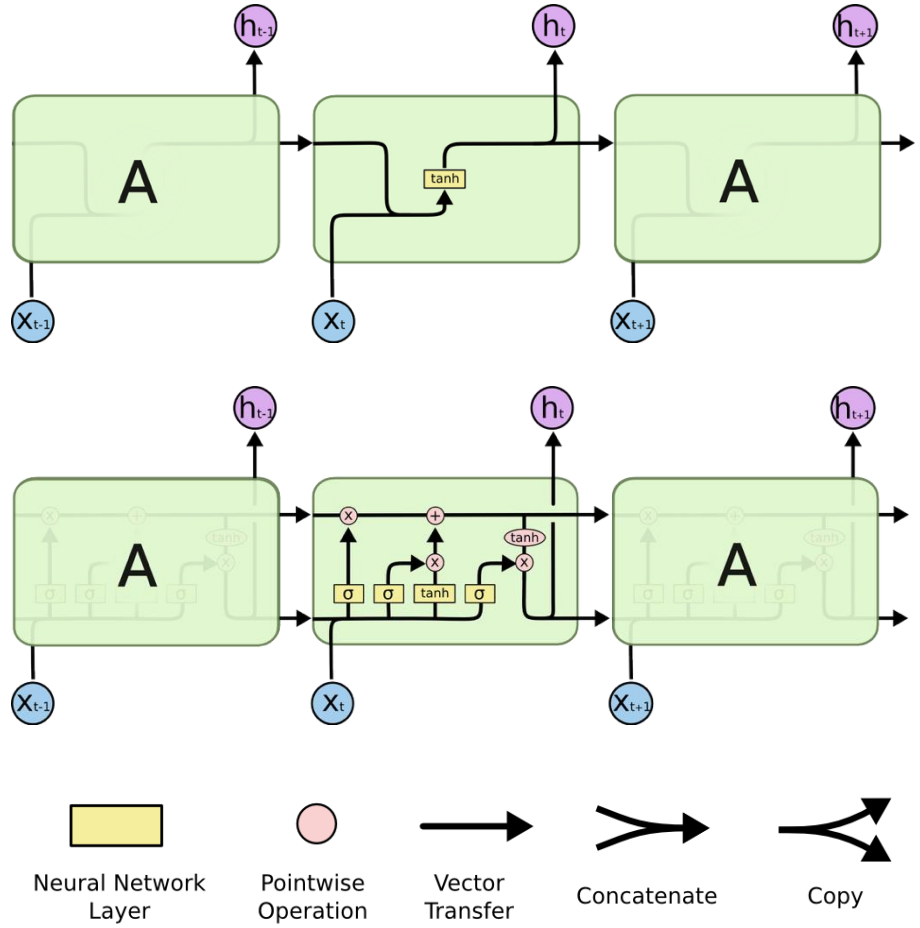
- LSTM mitigates the limitations highlighted previously with a vanilla RNN
  - Introduces a **memory cell** and **gates**
    - Controls what to remember, forget and output
- An LSTM memory cell contains 3 gates:
  - Input Gate
    - Controls which information is added to the memory cell
  - Forget Gate
    - Controls which information is removed from the memory cell
  - Output Gate
    - Controls which information is output from the memory cell

Vanilla RNNs and LSTM

- All RNNs can be viewed as a chain of repeating modules.

- In a vanilla RNN, each repeating module has a simple structure.
  - Typically, a single tanh layer.

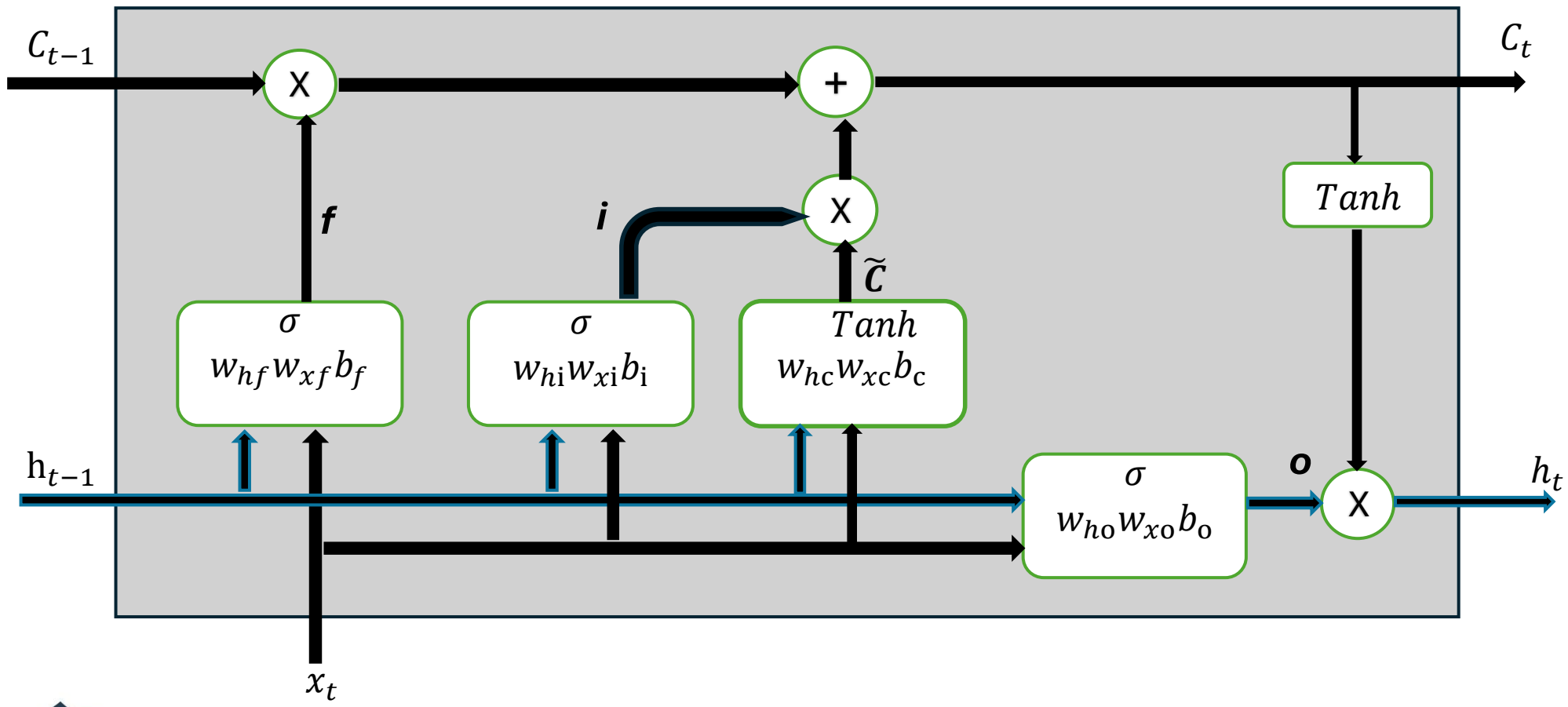
- LSTMs also follow this chain-like structure.
  - However, the repeating module in an LSTM is more complex.
  - Instead of a single layer, it consists of four interacting components (gates and cell operations).



Images from https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Pilot
UK AI Factory Antenna

LSTM Cell



Pilot
UK AI Factory Antenna

LSTM Cell continued

- Note the 3 gates: ***f*** (forget), ***i*** (input) and ***o*** (output)
- **Forget Gate** decides what information from the previous cell state to discard
  - $\boldsymbol{f_t} = \sigma(w_{xf}x_t + w_{hf}h_{t-1} + b_f)$
- **Input Gate** decides which values to update in the memory
  - $\boldsymbol{i_t} = \sigma(w_{xi}x_t + w_{hi}h_{t-1} + b_i)$
  - **Candidate state, $\widetilde{C}$**, uses tanh layer to extract new information from the current input (x_t) and the previous hidden state (h_{t-1})
    - $\widetilde{\boldsymbol{C_t}} = \tanh(w_{xc}x_t + w_{hc}h_{t-1} + b_c)$
  - **Cell State** is computed using
    - $\boldsymbol{C_t} = (f_t * C_{t-1}) + (i_t * \widetilde{C}_t)$
- **Output Gate** filters the cell state to produce the hidden state (h_t)
  - $\boldsymbol{o_t} = \sigma(w_{xo}x_t + w_{ho}h_{t-1} + b_o)$
  - $\boldsymbol{h_t} = o_t * \tanh(C_t)$

Pilot
UK AI Factory Antenna

Many Other RNNs

- Gated Recurrent Unit (GRU)
  - Simpler architecture than LSTM
  - Combines “forget” and “input” gates into a single “update” gate
  - Additionally, merges cell state and hidden state
- Many more
  - Depth-gated RNNs
  - Clockwork RNNs
  - DRAW
    - RNN for image generation
  - Etc.

RNNs: Pros

- Process sequence data over time
  - Order matters.
- Fast to train and lower memory footprint.
- Maintains a temporal context
  - Incorporate previous observations via hidden states.
  - LSTMs handles longer-term dependencies better than vanilla RNNs.
- RNNs/LSTMs can process sequences of different lengths.
  - Sentences with 5, 20, or even longer words.
  - Sensor recording lasting 20 seconds, 3 minutes, etc.
- Good for relatively small datasets
  - RNNs/LSTMs have fewer parameters than Transformer architectures.
- Good for streaming/online inferencing
  - Don't often need the entire sequence available at once.

RNNs: Cons

- Inherent sequential computation
  - Can **not** compute current hidden state, h_t, until the previous hidden, h_{t-1},state is computed.
    - Transformers, for example, can process tokens/words in parallel during training.
- Training data containing long sequences can be slow
  - E.g., training an LSTM network with sequences of length say 1,000 can be slow.
    - Transformers can do this faster due to parallel processing.
- Vanilla RNNs are vulnerable for vanishing/exploding gradients.
  - LSTMs mitigate this problem.
- Not suitable for long-range dependencies
  - E.g., language translation or speech recognition where context spans 1,000 words/tokens.

Pilot
UK AI Factory Antenna

Transformers

- Current main Large Language Models (LLMs) are based on the Transformer architecture; e.g.,
  - Generative Pre-trained Transformer (GPT) series from OpenAI
  - Open-source Llama series from Meta
  - Gemini from Google
  - Claude from Anthropic
- Fixes shortcomings of previous sequence data processing architectures like Recurrent Neural Networks (RNNs) used for language translations
  - Limitations with handling long range dependencies in a large dataset
    - Context could be lost in a large dataset
  - Each token is handled one at a time, **sequentially**
    - Leads to vanishing and exploding gradients

Pilot
UK AI Factory Antenna

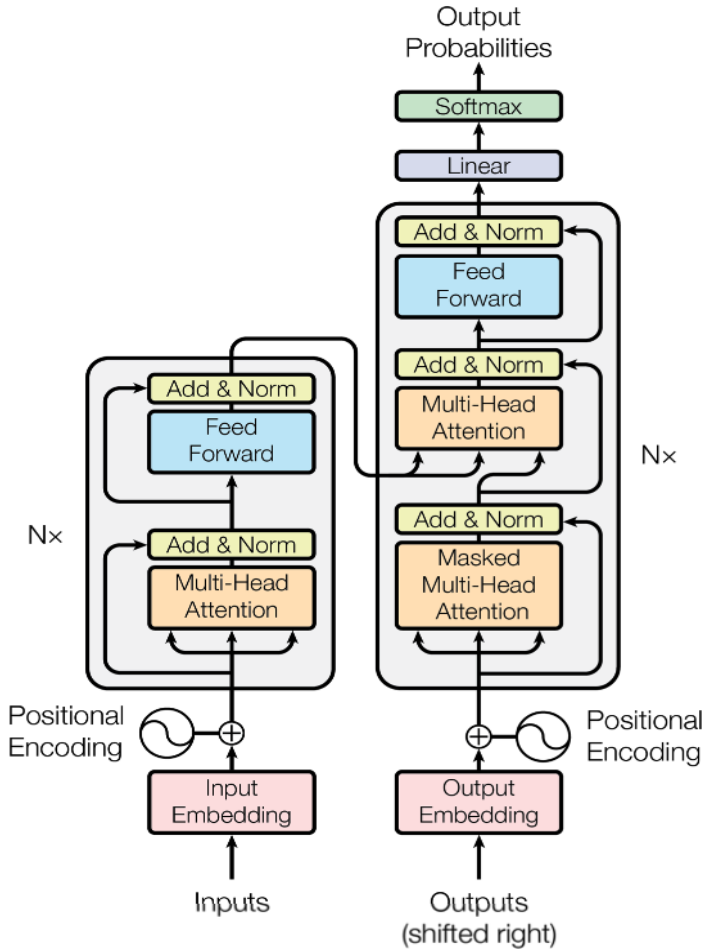
Transformers: Architecture

- Introduced in the landmark paper published in 2017 titled "Attention Is All You Need"

- Note the two main sections/blocks:
  - Encoder
  - Decoder

- A specific Transformer **model** may repeat Encoder and Decoder blocks certain number of times
  - Nx indicates this in the figure

- Has many important architectural components
  - Including the FeedForward NN that you have learnt earlier
  - Many components are referred to as layers in the paper



From the paper "Attention is All You Need", https://arxiv.org/pdf/1706.03762.pdf

Pilot
UK AI Factory Antenna

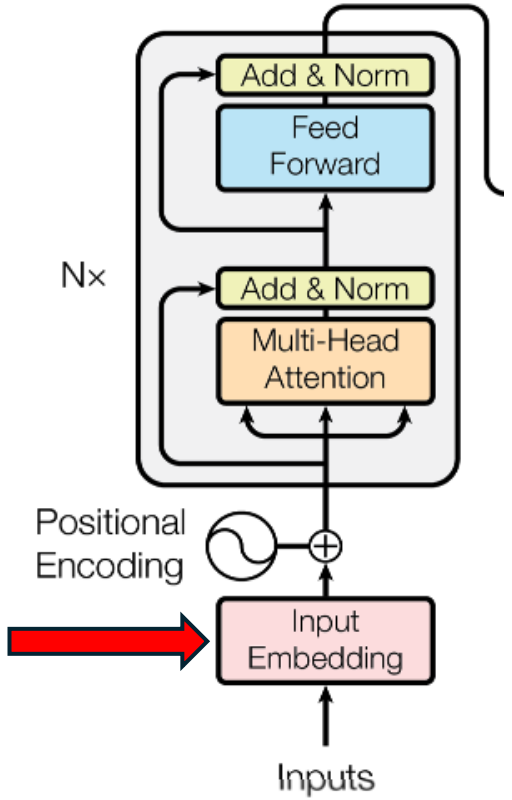
Transformers: Important Components

- **Input Embeddings**
  - Conversion of words in a sequence to their numerical representation
- **Positional Encoding**
  - positional encoding is used to provide information about the position of each word/token in the sequence
- **Self Attention**
  - allows each word in a sequence to attend to all other words in the same sequence
- **Multi-head Attention**
  - extends the self-attention mechanism by performing multiple attention operations in parallel, each focusing on different parts of the input sequence
- **Masked Multi-head Attention**
  - employed to prevent the model from attending to future words during training
  - necessary in text generation and chat applications
- **Feedforward Neural Network**
  - Used to perform non-linear transformations so that the model can learn complex patterns and features in the input data
- **Residual Connections**
  - used to mitigate the vanishing gradient problem and facilitate the training of deep Transformer architectures
- **Layer Normalisation**
  - applied to stabilize the training process and improve the convergence of the model

Pilot
UK AI Factory Antenna

Transformers: Input Embeddings

- Lets consider the sentence:
  "Attention is all you need"
- Above sentence has to be converted to numbers
  - Neural networks deals with numbers (not letters or words)
  - Need a mechanism to turn words in above sentence to numbers
  - Word embeddings:
    - Each word is mapped to a vector of numbers
      - E.g., "Attention" might become [0.3, 0.8. 0.5]
    - Multiple numbers are required to represent multiple meaning and relationships of a word
      - A single number can *not* represent all necessary symantic and contextual complexities in a language



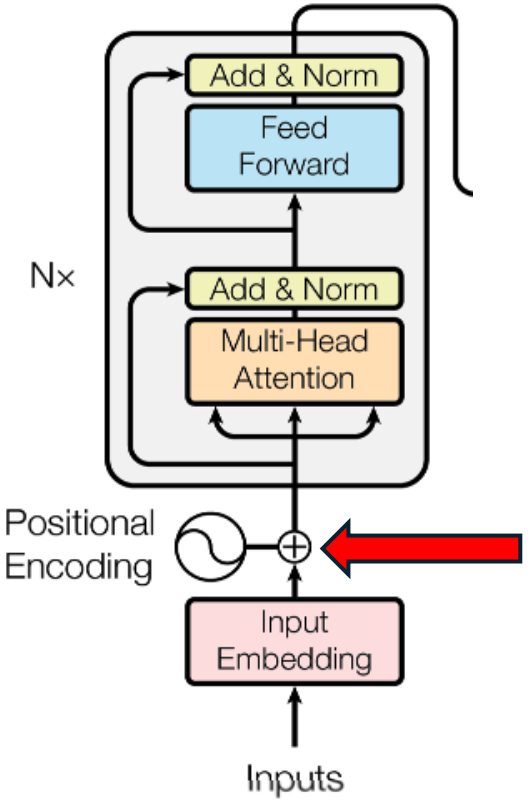
Pilot
UK AI Factory Antenna

Transformers: Positional Encoding

- This is basically labelling words with numbers to indicate position of each word in a sequence
  - So that model knows which word comes before and after a given word.
    - i.e., model knows the order of words
  - In the sequence "Attention is all you need"
    - "Attention" might be marked as position 1, "is" as position 2, etc.

- As the architecture shows these positional encodings are added to the input word embeddings that were described earlier
  - Each word embedding encode both meaning of the word and its position in the sequence

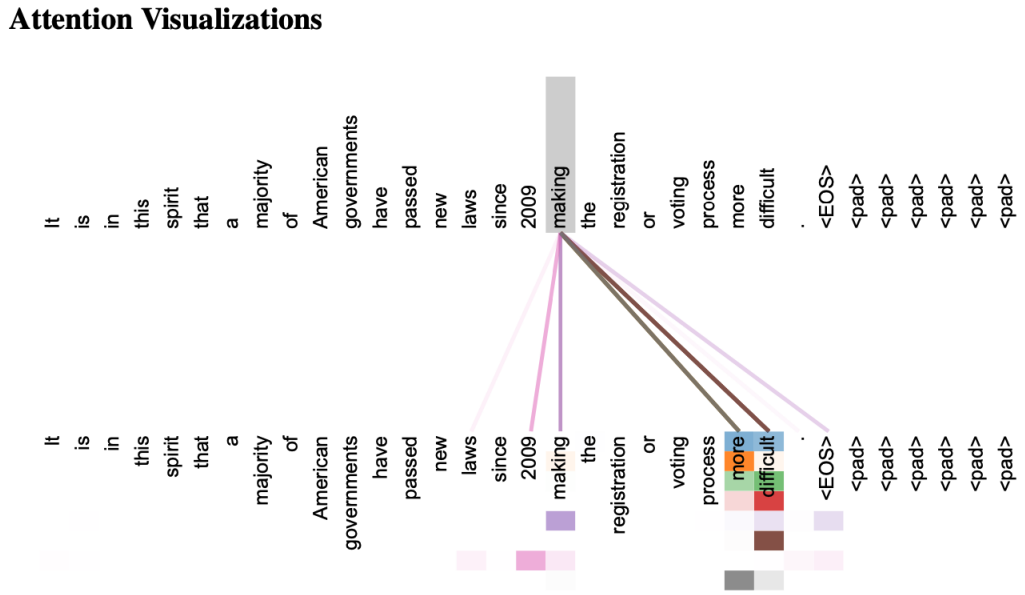
- Transformers use a sinusoidal function to create positional encoding
  - E.g., Sine or Cosine function
    - Provides a simple continuous functions with output beween -1 and 1
  - That is positional encodings are based on predefined functions and does not dependent on the content of the input sequence
    - A discrete function would affect model's generalizability; for example, ability to handle variable sequence lengths when inferencing with **unseen** data.



Pilot
UK AI Factory Antenna

Transformers: Self Attention

- Probably the most important concept as the title of the paper indicates
  - Idea: how does a word in a sentence is related to all other words in that sentence
- For each word, self attention assigns weights to the other words based on how important those words are to the current word
- Then a weighted sum is calculated based on above weights
  - this provides a context of how important are other words to the current word
- The weighted sum is used to update the representation of the current word, capturing its meaning in terms of other words in the sentence



From the paper “Attention is All You Need”, https://arxiv.org/pdf/1706.03762.pdf

Pilot
UK AI Factory Antenna

Self Attention: Query, Key and Value

- Query (*Q*), Key (*K*) and Value (*V*) are used to calculate Self Attention
- Lets assume current word is "need"
  - **The query is what is the relationship between the word "need" and other words in the sentence**
- Every word in the sentence has a Key
  - E.g., key for "need" could be position in the sentence.
  - Compare query of "need" with all other words in the sentence
  - Using the comparison, calculate how relevant each word's key to the query for "need"
    - i.e., calculate the **Attention Score**
    - Other words most relevant to this query will have higher attention scores
- Value is the **context** (or information) we want to associated with each word (e.g., "need")
  - For "need", calculation of the value may include words "all", "you" and "is"
  - Use the calculated relevance score (above) to determine how much importance to allocate for each word's value
  - Combine the values of all the words

Calculation of Self Attention

- Use the following formulae to calculate the Self Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

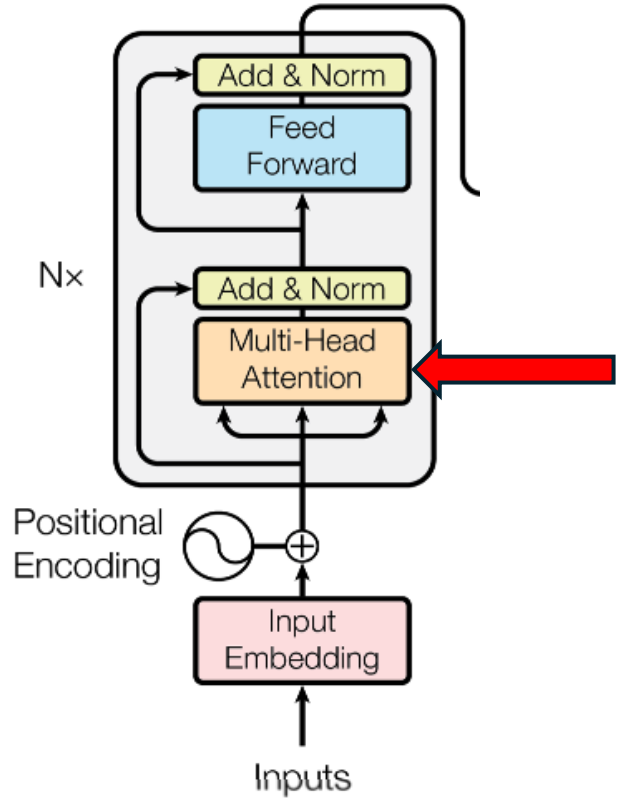
- Where *Q* is the Queries matrix, *K* is the Keys matrix and *V* is the Values matrix
  - Here we are handling all the queries simultaneously in a single matrix where each row is single query vector
- Note that the multiplication of $Q*K^T$ will emphasize words most important to the current query, *Q*, (e.g., word "Attention") based on all the keys, *K*, in a sequence.
  - i.e., this is how the **Attention Scores** are calculated
    - A higher attention score implies that the word represented by the key is more important to the query
- *d* is embeddings dimension (e.g., 512)
- Softmax function will return a probability distribution between 0 and 1, ensuring that all probabilities sum up to 1.
  - This is normalization of attention scores to create **Attention Weights**
  - Closer to 1 indicates more important a value in V to the current query
- *V* is values matrix where each row contains value representing a word in the sequence.
- Attention Weights are then multiplied by value vectors packed in *V*
  - This calculates the weighted sum of values in *V* where each weighted sum represent a context indicating level of importance of words in the sequence to the current query

Multi-head Attention

- Think of previous Self Attention as single-head Attention
- We can attend to different important aspects of a sentence in parallel by using multi-head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.



Multi-head attention as explained in the paper “Attention is All You Need”, https://arxiv.org/pdf/1706.03762.pdf

Pilot
UK AI Factory Antenna

Benefits of multi-head Attention

- Learning different relationships in parallel
  - Each head can learn to attend different aspects of an input sequence
  - This learning happens simultaneously
    - i.e., can make use of parallel processing
- Improved representation
  - Output of the multi-head attention contains a richer representation of an input sequence using different perspectives, learnt via different heads
- Flexibility
  - Number of heads in the model is a hyperparameter that can be tuned based on a specific task and required parallelism
    - E.g., bigger HPC platform could use a larger number of heads

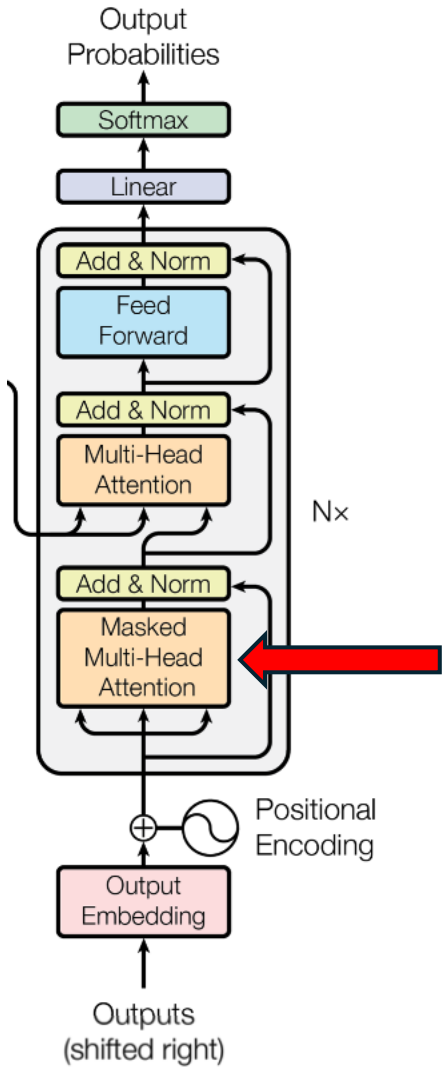
Pilot
UK AI Factory Antenna

Decoder: Masked Multi-Head Attention

- A special type of multi-head attention where future words are hidden whilst training
  - i.e., model must **not** be able to see future words
- Current predicted word is only based on previously predicted words
  - From the paper:

    "predictions for position *i* can depend only on the known outputs at positions less than *i*"
- This means the matrix values above the main diagonal need to be zero for:

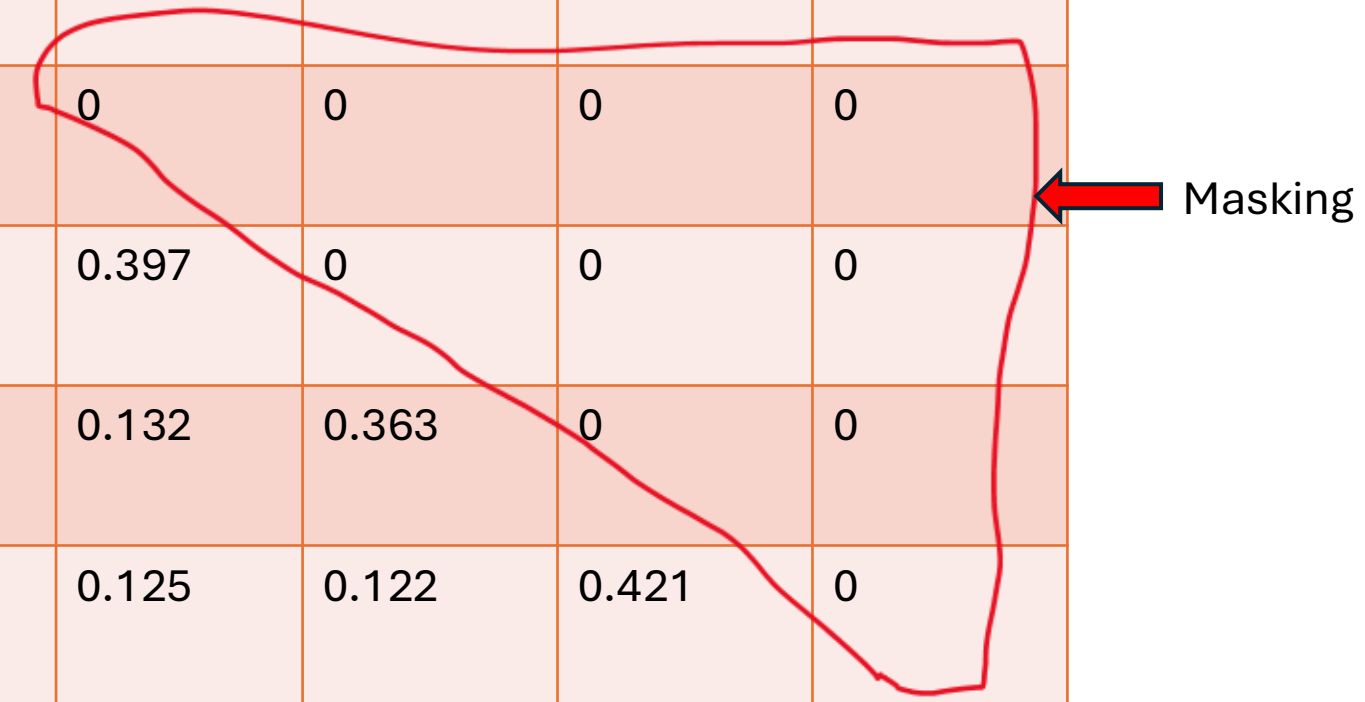
$$\text{softmax}(\frac{QK^T}{\sqrt{d_k}})$$



Pilot
UK AI Factory Antenna

An example masked matrix

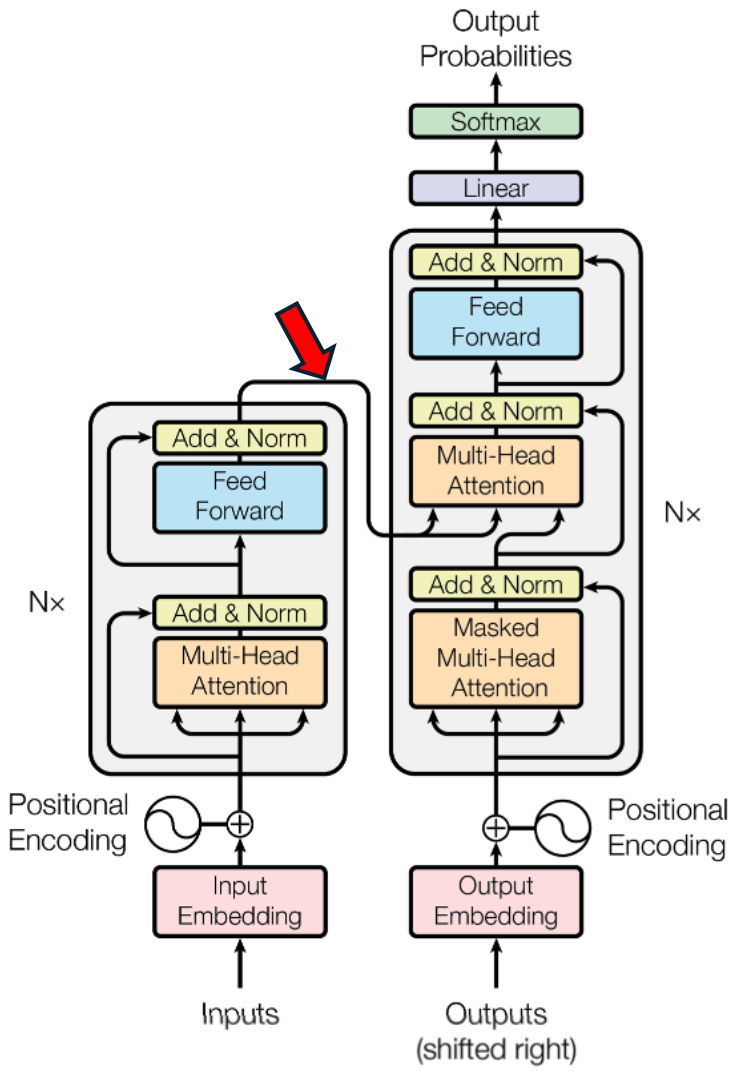
	Attention	is	all	you	need
Attention	0.336	0	0	0	0
is	0.124	0.397	0	0	0
all	0.166	0.132	0.363	0	0
you	0.127	0.125	0.122	0.421	0
need	0.122	0.189	0.101	0.276	0.312



Pilot
UK AI Factory Antenna

Cross Attention

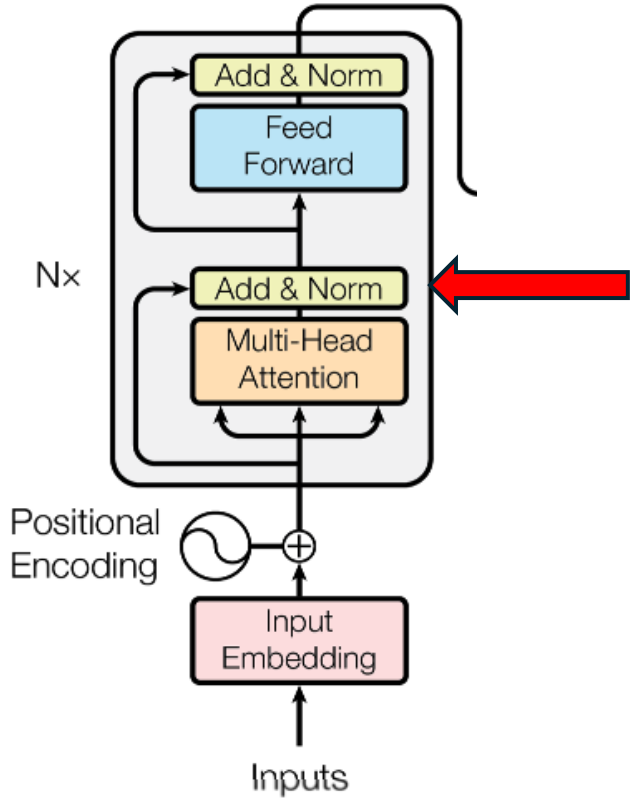
- Also referred to as encoder-decoder attention
- Necessary to understand dependencies between an input sequence and an output sequence
- Identifies semantic relationships between words in different languages
- Handles variable lengths in input and output sequence
  - Since this focus only on important words and their relationships
- Queries from the Decoder is linked with Keys and Values from the Encoder



Pilot
UK AI Factory Antenna

Residual Connections

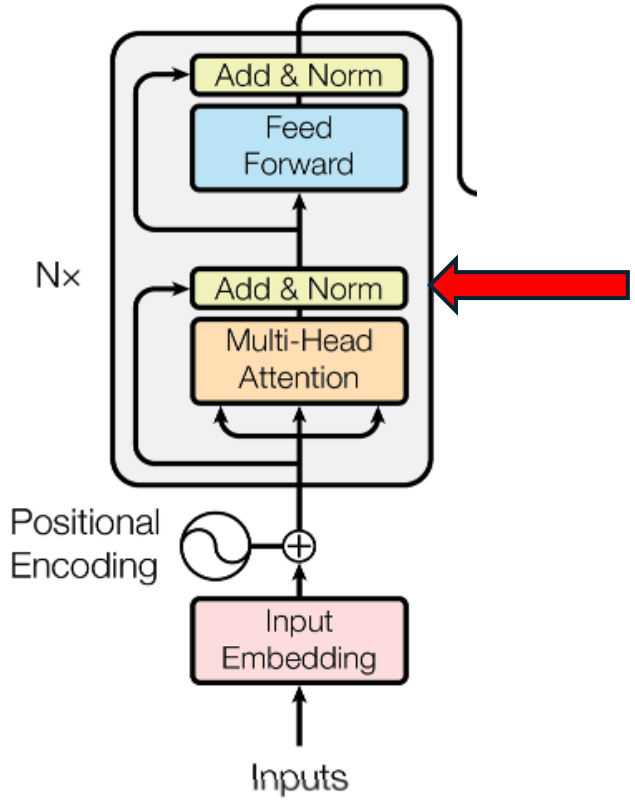
- Also referred to as skip connections
- Inputs are directly **added** to the output of the component
- Used in other previous neural network architectures
- Transformer adds residual connections around each component
- Addresses the vanishing gradient problem
  - Provides a direct path for gradients flow during training
  - Helps in training deep neural networks where gradients have to propagate through many layers and components
- Mitigate issues arising when a component not learn from data
  - As inputs simply pass to the next component without any changes



Pilot
UK AI Factory Antenna

Layer Normalisation

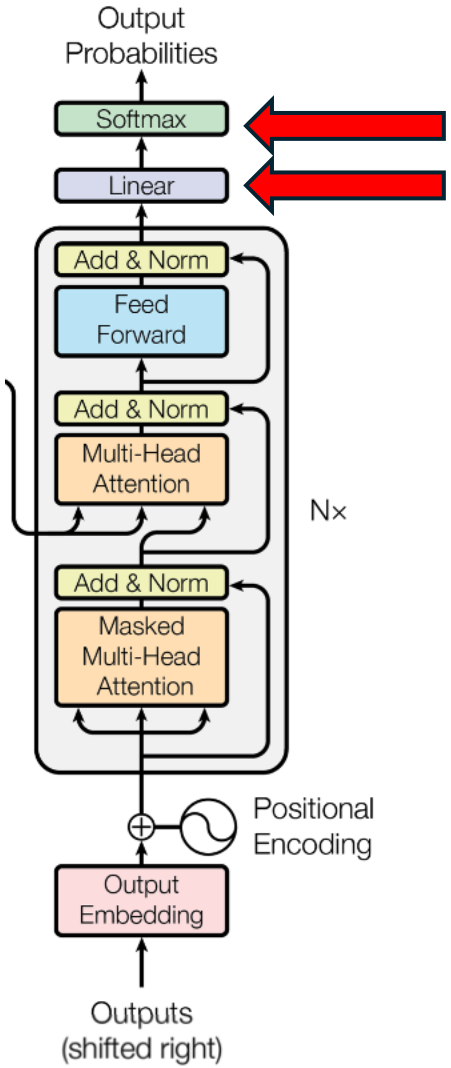
- Normalisation is the process where numerical values are scaled so that they have a mean of 0 and standard deviation of 1
  - Makes the numerical values easier to work with during training
- Layer normalization is where normalization applied to each layer (i.e., component) in the architecture
  - Applied to the activations of neurons in the layer
  - Leads to stable and efficient training



Pilot
UK AI Factory Antenna

Final Linear Component + Softmax

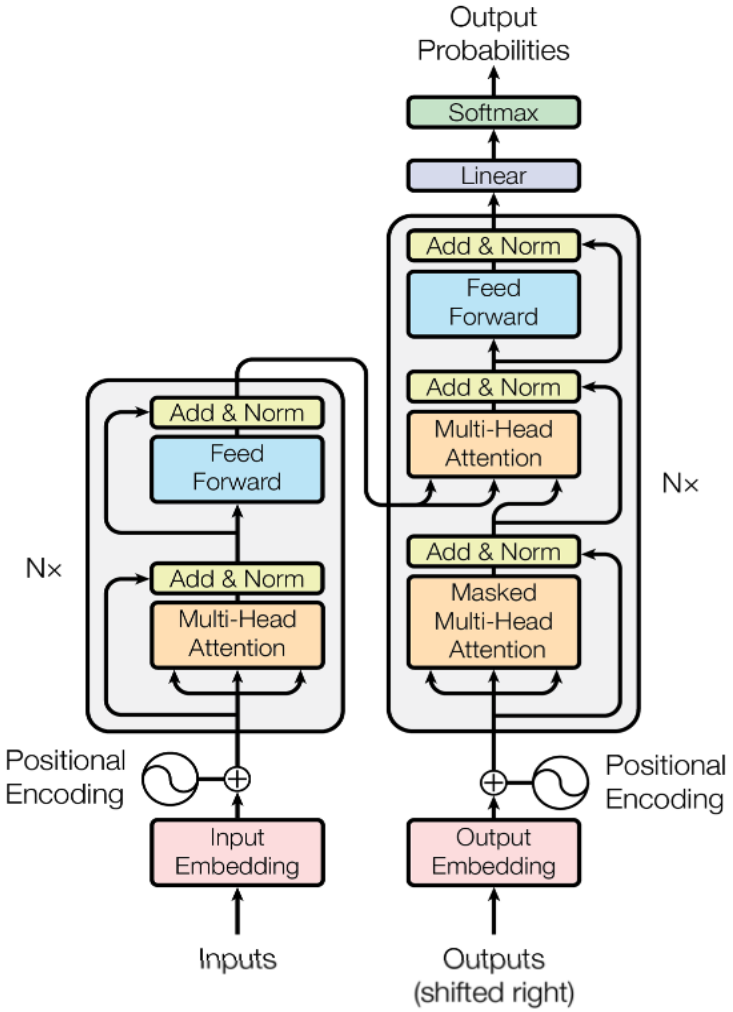
- In many Transformer models the final layer performs both the linear transformation and softmax function
- Linear component projects the encoded sequence to the vocabulary
  - Act as the translator that produces the meaningful sequence based on the probabilities for the individual words
- Softmax function produces the output word sequence based the on predicted probabilities



Pilot
UK AI Factory Antenna

An example: training a language translation

- Inputs: <SOS> I love PyTorch <EOS>
  - This is fed into the Encoder (left side)
- Outputs:<SOS> J aime PyTorch
  - This is fed into the Decoder (right side)
- Sequence lengths are generally fixed
  - E.g., 1000 words (so requires padding tokens)
- Inputs and outputs are fed through the model simultaneously
  - First they are converted to embeddings and
  - Positional encodings are added
  - Then goes through the main components described above
    - Applying multi-head attention, cross-attention, FNNs, Linear layer and finally applying softmax function to predict the output sequence
    - Note that after each main component, layer normalization is applied as well as addition of residual connections
  - All this happens in one time step when training
    - Thanks to all the parallel processing enabled by the transformer architecture
- At inferencing each word will be predicted one time step at a time
  - Outputting the word with the highest probability



Pilot
UK AI Factory Antenna

Transformers: using PyTorch

```
class TransformerModel(nn.Module):
 # Create the Transformer architecture to use
 def __init__(self, input_dim, output_dim, hidden_dim, num_layers=1, num_heads=1):
  super(TransformerModel, self).__init__()
  #Embedding layer to convert input words to dense vectors of fixed size (hidden_dim)
  self.embedding = nn.Embedding(input_dim, hidden_dim)
  #Main Transformer block which utilises PyTorch Transformer implementation
  self.transformer = nn.Transformer(
   #Hidden dimension
   d_model=hidden_dim,
   #number of attention heads
   nhead=num_heads,
   num_encoder_layers=num_layers,
   num_decoder_layers=num_layers,
  )
  #Final linear layer
  self.fc_out = nn.Linear(hidden_dim, output_dim)
```

Transformers: using PyTorch - continued

```
class TransformerModel(nn.Module):
  ...
 #Forward pass of the Transformer model that takes source and target sentences
 def forward(self, src, trg):
  #Convert the source and target senetences to embeddings
  src = self.embedding(src)
  trg = self.embedding(trg)
  #Pass the source and taget embedings through the Transformer model
  output = self.transformer(src, trg)
  #Final output is produced by going through a fully connected layer that
  #produces predictions based on probabilities for each word in the target vocabulary.
  output = self.fc_out(output)
  return output
```

Transformers: Pros

- Addresses limitations of RNNs and Long Short-Term Memory (LSTM) networks designed to solve sequence prediction problems; e.g.,
  - Language Translation
  - Chat applications
- Transformer architecture can capture long range dependencies more effectively without suffering gradient related issues
  - Self-attention mechanism allows the architecture to capture context from the entire input sequence
- Positional encoding maintains information about position of each token in a sequence
- Suitable for parallel processing
  - A Transformer model can process all tokens (e.g., words) in an input sequence simultaneously
- Highly scalable
  - Can handle large dataset (e.g., an entire book)
  - Models can have billions or trillions of parameters
    - E.g., Suitable for utilising HPC infrastructure with large number of GPUs

Pilot
UK AI Factory Antenna

Transformers: Cons

- Transformers use probability distribution, for example, when predicting the next word in a language translation task
  - Therefore, what's written may not be correct (hallucinations)
- Models tend to be very large
  - Many billions to trillions of parameters models now exists
  - Require expensive large AI-accelerated HPC hardware infrastructure
    - Specialist engineering skills are required to make effective use of such HPC resources
    - Makes less accessible to many researchers and engineers
- Typically require a large amount of data due to the large amount of parameters present
  - Fine tuning on small datasets may not be sufficient
- Transformers operate within a fixed context window
  - Determined by the maximum sequence length
    - Affects the understanding of the context if the input sequence is larger than the maximum context window
- Pre-training overfitting may occur
  - A model may be overfit to pre-trained data and so may not performed well when fine-tuned with data for another application domain

Pilot
UK AI Factory Antenna

DNNs: Key Takeaways

- In DNNs deeper layers learn more abstract and higher-level representations by combining representations learned in earlier layers.
  - E.g., pixels→edges→shapes→object parts→objects
  - Increase the **depth** to recognise higher representations
- More neurons in a layer increase representational capacity
  - E.g., in earlier layers of a CNN -> different filters/neurons learn to respond to different visual patterns such as horizontal, vertical or diagonal edges.
  - Increase the **width** to increase the representational capacity.
- RNNs/LSTM suitable for short sequences
  - Particularly suitable for resource constrained environments.
  - Long range dependencies remain a challenge.
  - Difficult to parallelise.
- Transformers are suitable for long sequences
  - Current main open and closed-source LLMs are based on the Transformer architecture.
  - Central concept is Self Attention.
    - Used via multi-head and masked multi-head attention
  - Typically require a large amount of data due to the large amount of parameters present.
  - The architecture is highly suitable for parallel processing
    - A significant advantage over previous architectures like RNNs.

Keep in Touch with Pilot - UKAIFA

- If you're interested in our future training courses or the wider project, please complete the following short form:

  https://forms.cloud.microsoft/pages/responsepage.aspx?id=sAafLmkWiUWHiRCgaTTcYSwVxa7BviBKn8Png9rfMoFURE1DSzFVSTQ3UlZYRjhIQVdUSzYxOVFLMS4u&route=shorturl

- You can also use this form to provide feedback on today's webinar. We're always happy to receive feedback.
- Find out more about the project at: www.pilot-ukaifa.ai

UKAIFA Course Feedback - Introduction to DNNs - Webinar - 2026-09-07

Pilot
UK AI Factory Antenna

Acknowledgements and Reuse

- Most of this presentation has been created by Dr Charaka Palansuriya, EPCC, The University of Edinburgh as part of the Pilot-UKAIFA project
  - It draws some elements for courses previously delivered on EPCC's *MSc in High Performance Computing with Data Science* with some slide content originally developed by others at EPCC including Dr Adam Carter and Prof Adrian Jackson.

Pilot
UK AI Factory Antenna

Introduction to Deep Neural Networks (DNNs)

- Thank you for Listening!
- Questions & Discussions